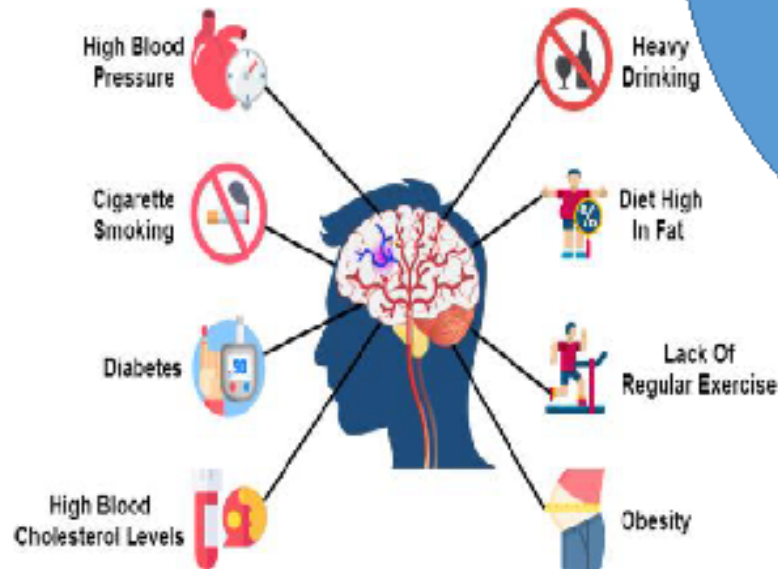


STROKE PREDICTION DATASET
DATE : NOV .04, 2023
BY : NEGUSU WORKU GEBRMICHAEL
INSTRUCTOR:
MR. VIJAY KUMAR



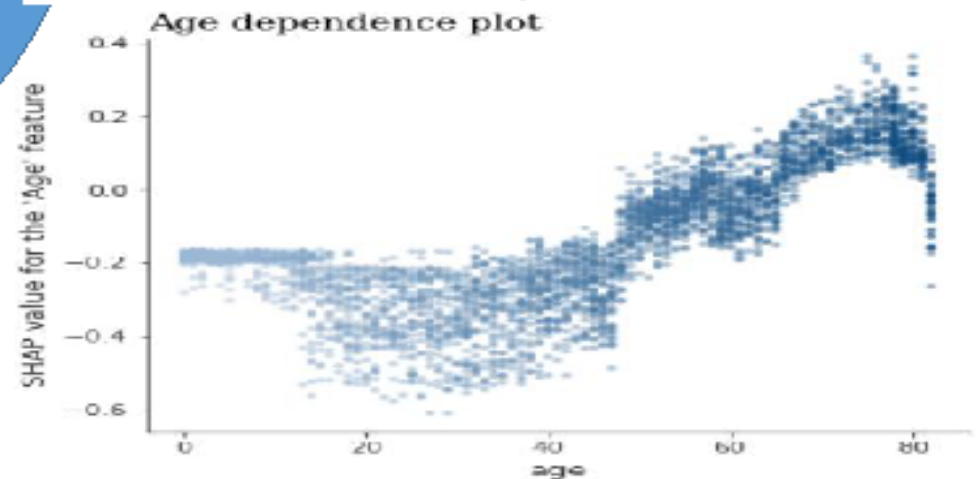
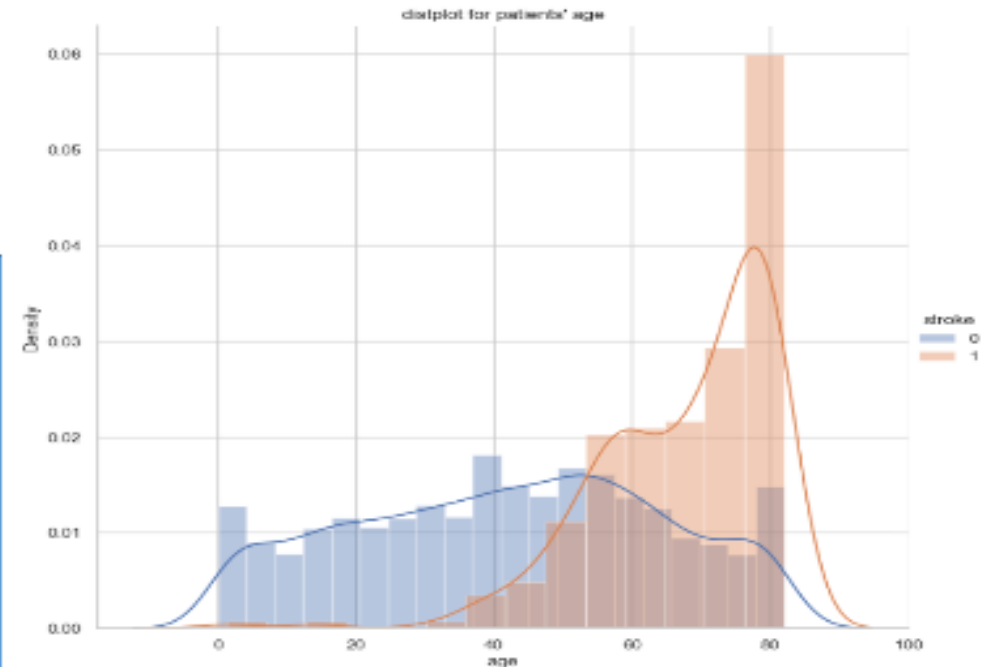
Stroke Prediction, n=5111, Prepared by Negusu Worku GEBRMICHAEL, November, 04, 2023

According to the World Health Organization (WHO) stroke is the 2nd leading cause of death globally, responsible for approximately 11% of total deaths.



n=5111 Obs

- Older patients are more likely to suffer a stroke than a younger patient,
- BMI and Age were positively correlated
- Female in private companies and self employment are easier to get stroke,
- More stroke cases are observed urban than rural),
- People working in private companies in urban areas are more likely to have strokes than those working in private companies in rural areas.



PRESENTATION OUTLINE

- 1. Executive Summary,
- 2. Problem Statement,
- 2. Hypothesis test,
- 3. Methods
 - Feature Engineering
 - Univariate
 - Bivariate
 - T-Test
 - Chi-square
 - Regression
 - Linear regression for prediction
 - Cross Validation
- Conclusion

EXECUTIVE SUMMARY

Executive Summary

A total of **n=5110** cases of stroke data was obtained from the kaggle web. The dataset has **n=12 variables** (columns). Among the observations **n=2115 (41.4%)** of them were found to be males and **n=2995 (58.6%)** are females. The age range is **from 0.08 - 82** years and the mean is **43.22**. The residence type among the sample size shows that **49.2% of them were from rural and 50.8% urban**. The marital status among the sample size shows **not-married n=1757 (34.4%), Yes married n=3353 (65.6%)**. The distribution of work type among the case shows children **n=687(13.4%), government job n=657(12.9%), never worked n=22(0.4%), private n=2925(57.2%)** and **self-employed n=819(16.0%)**. Hypertension distribution among the case shows that **n=4612(90.3%) are no** and **n=498(9.7%) yes**. Smoking status among the case shows, **n=885(17.3%), never n=1892(37.0%), smokers n=789(15.4%), and unknown n=1544(30.2%)**. Moreover, the average glucose level mean shows **106.14, range is 216.62**, minimum is 55.12 and maximum is 271.74. The heart disease availability also shows, **n=4834(94.6%) no** and **n=276(5.4%) are Yes**.

PROBLEM STATEMENT

- According to the World Health Organization (WHO), stroke is the 2nd leading causes of death globally responsible for approximately 11% of the total deaths.
- Based on the collected data n=5110 observations, data analysis conducted to isolate **which factor aggravated** the stroke among the cases. Prior to the analysis, the methods of data analysis techniques used to different layers including : behavioral, environmental, personal and chronic characteristics using the statistical methods of univariate, bivariate, chi-square test, T-Test, linear and multivariate analysis,
- Do behavioral level characteristics like smoking and BMI exposed to stroke ?
- Will work type exposed to Stroke ?
- Do personal and chronic diseases aggravate the situation of stroke like heart diseases, hypertension and average glucoses level ?

HYPOTHESIS TEST

I. Behavioral level

1. Smoking
2. Body mass index

II. Environmental level

1. Residence level (Urban or Rural

III. Personal Characteristics level (natural level)

1. Age
2. Sex
3. Marital Status

IV. Chronic level disease development (Internal)

- 1. Heart diseases
- 2. Hypertension
- 3. Average glucose level (AGL)

INTRODUCTION

<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>

According To The World Health Organization (WHO) Stroke Is The 2nd Leading Cause Of Death Globally, Responsible For Approximately 11% Of Total Deaths.

This Dataset Is Used To Predict Whether A Patient Is Likely To Get Stroke Based On The Input Parameters Like Gender, Age, Various diseases, and smoking status. Each row in the data provides relevant information about the patient.

Column	Description
id	unique identifier
gender	"Male", "Female" or "Other"
age	age of the patient
hypertension	0 = none-hypertension, 1 = hypertension
heart_disease	0 = none- heart diseases, 1 = heart disease
ever_married	"No" or "Yes"
work_type	"Children", "Govt_jov", "Never_worked", "Private" or "Self-employed"
Residence_type	"Rural" or "Urban"
avg_glucose_level	average glucose level in blood
bmi	body mass index
smoking_status	"formerly smoked", "never smoked", "smokes" or "Unknown"

DATA INFO

```
dataset = pd.read_csv('healthcare-dataset-stroke-data.csv')
```

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   id                    5110 non-null  int64  
 1   gender                5110 non-null  object  
 2   age                   5110 non-null  float64 
 3   hypertension          5110 non-null  int64  
 4   heart_disease         5110 non-null  int64  
 5   ever_married          5110 non-null  object  
 6   work_type             5110 non-null  object  
 7   Residence_type        5110 non-null  object  
 8   avg_glucose_level     5110 non-null  float64 
 9   bmi                   4909 non-null  float64 
10   smoking_status        5110 non-null  object  
11   stroke                5110 non-null  int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

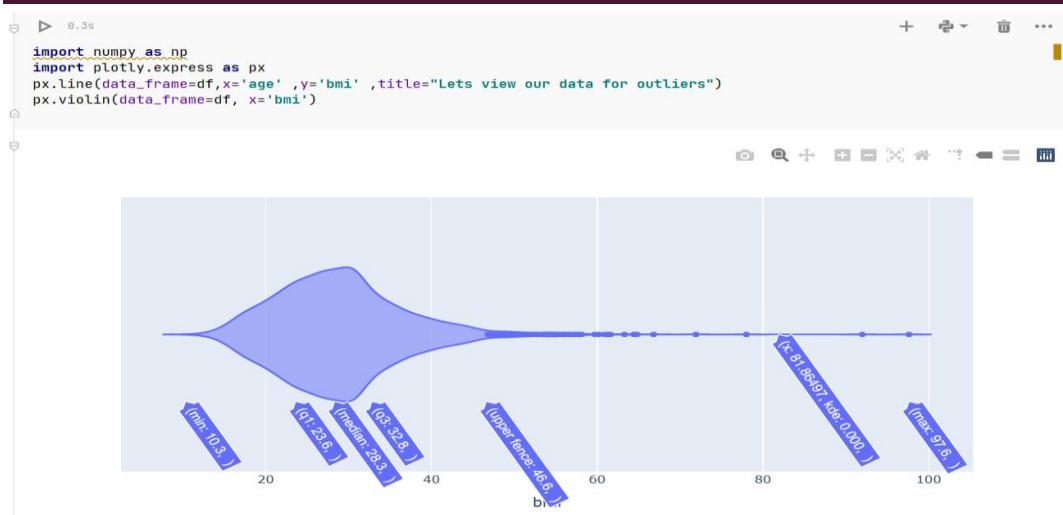
```
import numpy as np
dataset.describe()
```

	id	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	4909.000000	5110.000000
mean	36517.829354	43.226614	0.097456	0.054012	106.147677	28.893237	0.048728
std	21161.721625	22.612647	0.296607	0.226063	45.283560	7.854067	0.215320
min	67.000000	0.080000	0.000000	0.000000	55.120000	10.300000	0.000000
25%	17741.250000	25.000000	0.000000	0.000000	77.245000	23.500000	0.000000
50%	36932.000000	45.000000	0.000000	0.000000	91.885000	28.100000	0.000000
75%	54682.000000	61.000000	0.000000	0.000000	114.090000	33.100000	0.000000
max	72940.000000	82.000000	1.000000	1.000000	271.740000	97.600000	1.000000

LIBRARY

```
# library
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import math
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn import metrics
from statsmodels.graphics.gofplots import qqplot
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import PowerTransformer
import plotly.express as px
import scipy.stats as stats
import os
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import classification_report, confusion_matrix, f1_score
from sklearn.metrics import classification_report
from scipy.stats import chi2_contingency
import warnings
warnings.filterwarnings('ignore')
import pingouin as pg
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
```

DATA CLEANING- OUTLIERS



```
def traditional_outlier(df,x):
    q1 = df[x].quantile(.25)
    q3 = df[x].quantile(.75)
    iqr=q3-q1
    df['Traditional'] =np.where(df[[x]]<(q1-1.5*iqr),-1,
                                np.where(df[[x]]>(q3+1.5*iqr),-1,1))
    return df
```

```
traditional_outlier(df,'bmi')
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke	age_range	Traditional
0	Male	67.0	0	1	Yes	Private	Urban	228.69	36.600000	formerly smoked	1	[20, 85)	-1
1	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	30.541211	never smoked	1	[20, 85)	1
2	Male	80.0	0	1	Yes	Private	Rural	105.92	32.500000	never smoked	1	[20, 85)	1
3	Female	49.0	0	0	Yes	Private	Urban	171.23	34.400000	smokes	1	[20, 85)	1
4	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.000000	never smoked	1	[20, 85)	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
5105	Female	80.0	1	0	Yes	Private	Urban	83.75	30.541211	never smoked	0	[20, 85)	1
5106	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.000000	never smoked	0	[20, 85)	1
5107	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.600000	never smoked	0	[20, 85)	1
5108	Male	51.0	0	0	Yes	Private	Rural	166.29	25.600000	formerly smoked	0	[20, 85)	1
5109	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.200000	Unknown	0	[20, 85)	1

5110 rows × 13 columns



```
traditional_outlier(df,'avg_glucose_level')
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke	age_range	Traditional
0	Male	67.0	0	1	Yes	Private	Urban	228.69	36.600000	formerly smoked	1	[20, 85)	-1
1	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	30.541211	never smoked	1	[20, 85)	-1
2	Male	80.0	0	1	Yes	Private	Rural	105.92	32.500000	never smoked	1	[20, 85)	1
3	Female	49.0	0	0	Yes	Private	Urban	171.23	34.400000	smokes	1	[20, 85)	-1
4	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.000000	never smoked	1	[20, 85)	-1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
5105	Female	80.0	1	0	Yes	Private	Urban	83.75	30.541211	never smoked	0	[20, 85)	1
5106	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.000000	never smoked	0	[20, 85)	1
5107	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.600000	never smoked	0	[20, 85)	1
5108	Male	51.0	0	0	Yes	Private	Rural	166.29	25.600000	formerly smoked	0	[20, 85)	1
5109	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.200000	Unknown	0	[20, 85)	1

5110 rows × 13 columns

DATA CLEANING- UNIQUE VALUE & MISSING VALUE

(1) Drop ID and Replace Other Gender with Mode

```
df=dataset.drop(columns=['id'], inplace=False)
```

```
dataset['gender'].value_counts()
```

```
Female    2994  
Male      2115  
Other         1  
Name: gender, dtype: int64
```

```
print(dataset['gender'].mode())  
dataset['gender'].replace('Other', 'Female', inplace=True)
```

```
0    Female  
dtype: object
```

```
dataset['gender'].value_counts()
```

```
Female    2995  
Male      2115  
Name: gender, dtype: int64
```

(2) Filling Missing Values with 3 Mean BMI Age-wise

```
df['bmi'].isna().mean() * 100
```

```
0.0
```

```
df["age_range"] = pd.cut(df.age, bins=[0,2,20,85], right=False)
```

```
mean_df=df.groupby(["age_range"]).bmi.mean()  
print(mean_df)  
means=mean_df.agg(list) # converts all 3 means to python list  
for index,row in df.iterrows():
```

```
    age=row["age"]  
    if age>=0 and age<2:  
        row.replace(np.nan,means[0],inplace=True)  
  
    elif age>=2 and age<20:  
        row.replace(np.nan,means[1],inplace=True)  
  
    else:  
        row.replace(np.nan,means[2],inplace=True)  
    df.loc[ index] =row
```

```
age_range  
[0, 2)      18.478070  
[2, 20)     22.451205  
[20, 85)    30.541211  
Name: bmi, dtype: float64
```

NORMALITY TEST

Ist Step : Check the Normality

```
# defining numerical variables in a dataframe num_at
num_attribute = ['age', 'avg_glucose_level', 'bmi']
df_nums = df[num_attribute]
df_nums.head()
```

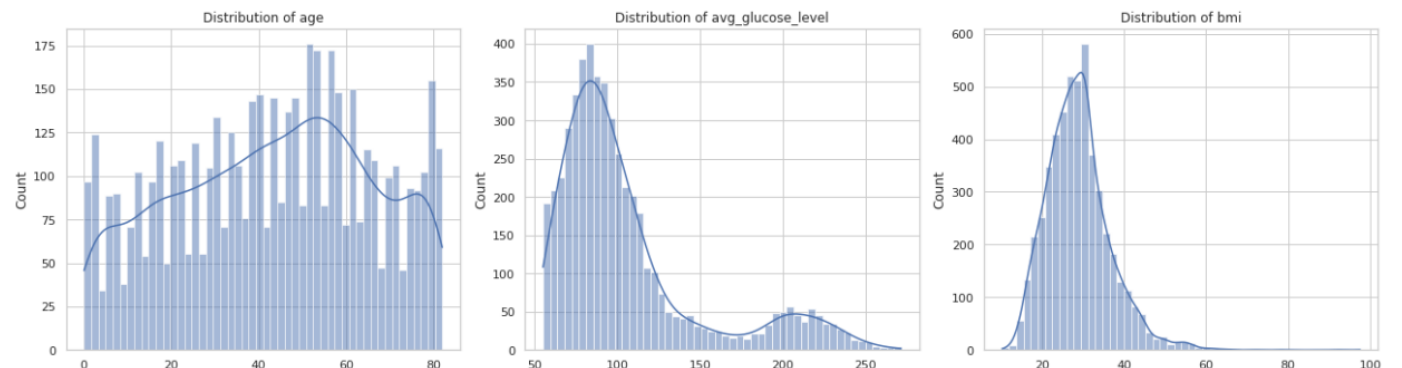
	age	avg_glucose_level	bmi
3295	0.08	70.33	16.9
1614	0.08	139.67	14.1
3618	0.16	114.71	17.4
4021	0.16	109.52	13.9
3968	0.16	69.79	13.0

```
#Layer 1 Normality Test
# Statistical Analysis on Contineous Numerical Features Asma
```

```
fig = plt.figure(1, (18, 5))
```

```
for i, attribute in enumerate(num_attribute):
    ax = plt.subplot(1, 3, i+1)
    # sns.displot(data=df_nums, x=attribute, kde=True)
    sns.histplot(data=df_nums, x=attribute, kde=True, bins=50)
    ax.set_xlabel(None)
    ax.set_title(f'Distribution of {attribute}')
    plt.tight_layout()
plt.show()
```

IF you see the graph below , only age is normally distributed. It can be seen that indeed the distribution is very skewed to the le;



NORMALITY TEST

2nd Step : Check the Log-normal

*#A log-normal distribution is a distribution of a random variable whose
Checking whether these features are log-normal or not.*

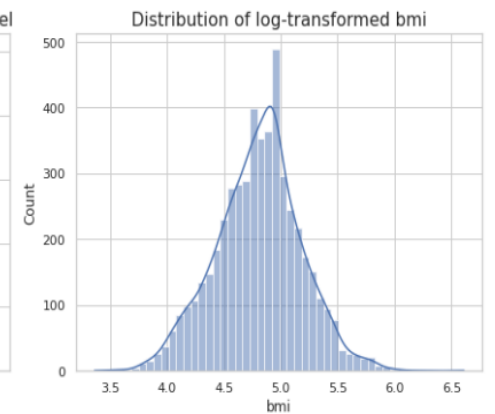
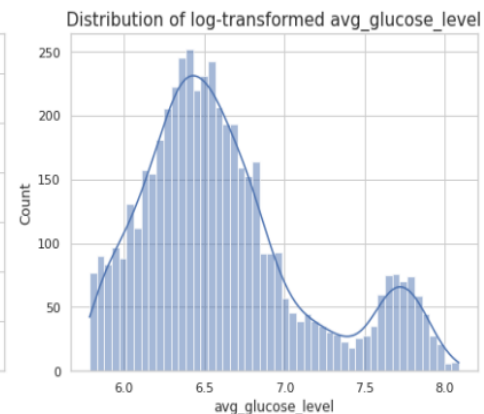
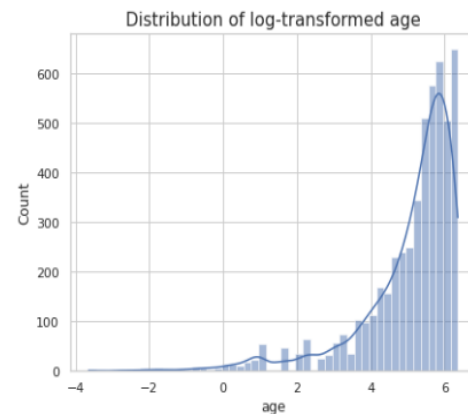
```
df_log = np.log2(df_nums)
df_log.head()
```

	age	avg_glucose_level	bmi
3295	-3.643856	6.136068	4.078951
1614	-3.643856	7.125878	3.817623
3618	-2.643856	6.841847	4.121015
4021	-2.643856	6.775051	3.797013
3968	-2.643856	6.124948	3.700440

```
1.3s
fig = plt.figure(1, (18, 5))

for i, attribute in enumerate(num_attribute):
    ax = plt.subplot(1, 3, i+1)
    ax.set_title(f'Distribution of log-transformed {attribute}', size=15)
    sns.histplot(data=df_log, x=attribute, kde=True, bins=50)
    plt.tight_layout()
plt.show()

# age is left skewed, while avg_glucose_level is also not log normal
# bmi has become almost entirely normal distribution after the log transformation.
# Higher peak iin the middle is because mean imputation
```



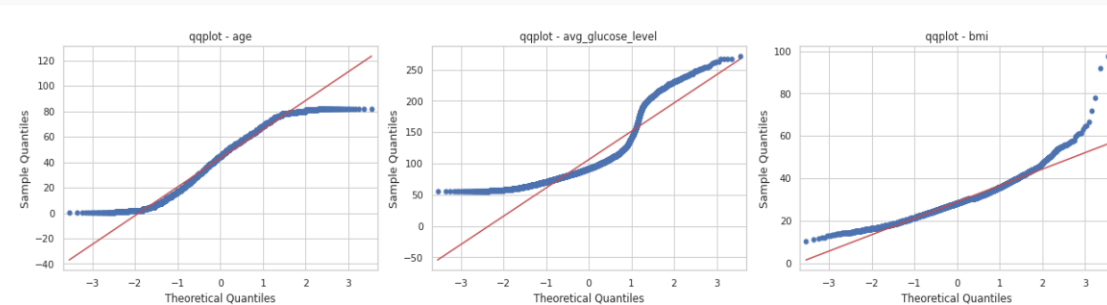
NORMALITY TEST

3rd Step : Log Transformation

```
from statsmodels.graphics.gofplots import qqplot
# quantile-quantile plots on original data
fig = plt.figure(1, (18, 8))

for i, attribute in enumerate(df_nums):
    ax = plt.subplot(2, 3, i+1)
    qqplot(df_nums[attribute], line='s', ax=ax, markersize=5)
    ax.set_title(f'qqplot - {attribute}')
    plt.tight_layout()
plt.show()
```

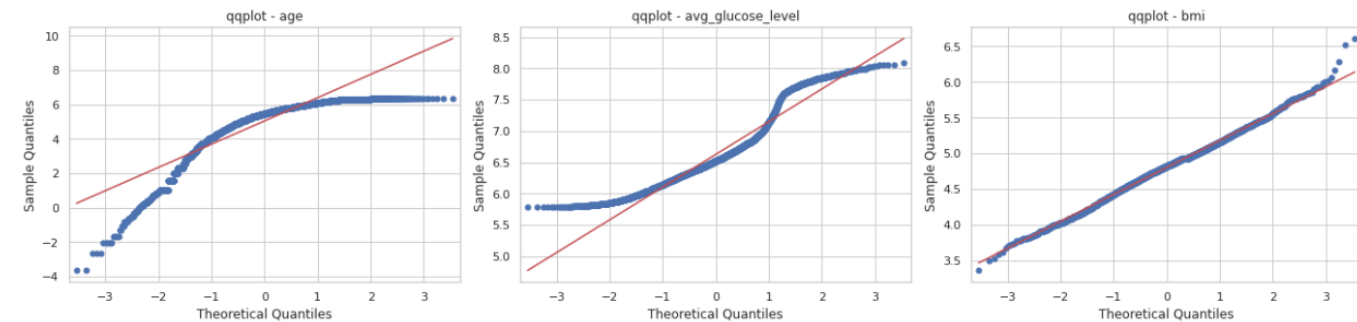
#visualization before transformation
#age is similar to before transformation, while avg_glucose_level has become more normal-like but still far from ideal.
#bmi has become almost entirely normal distribution after the power transformation.



```
fig = plt.figure(1, (18,8))
```

```
for i, num_feature in enumerate(df_nums):
    ax = plt.subplot(2,3,i+1)
    sm.qqplot(df_log[num_feature], line='s', ax=ax, markersize=5)
    ax.set_title(f'qqplot - {num_feature}')
    plt.tight_layout()
plt.show()
```

visualization after applying log transformation



NORMALITY TEST

4th Step : Power Transformation

```
# Power transformer by default takes yeo-johnson method to make distributed
from sklearn.preprocessing import PowerTransformer
```

```
pt = PowerTransformer()
transformed_df = pt.fit_transform(df_nums)
transformed_df = pd.DataFrame(transformed_df, columns=df_nums.columns)
transformed_df.head()
```

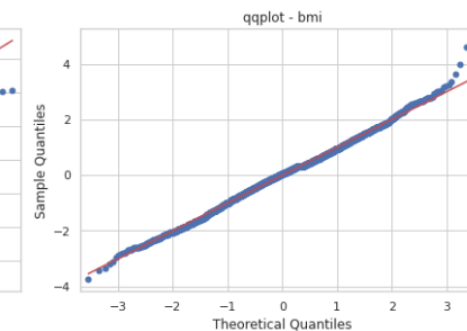
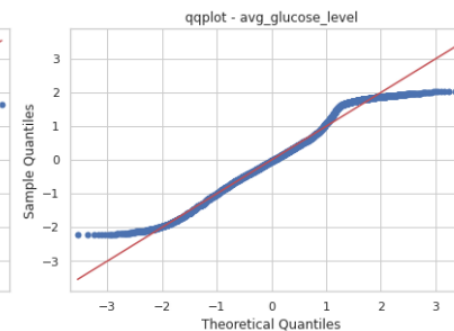
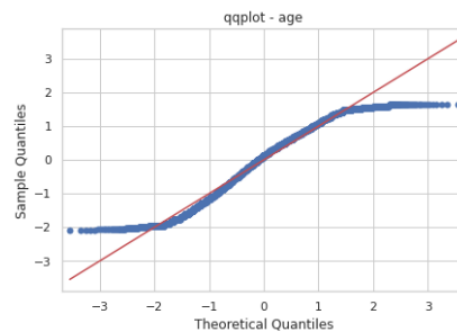
```
fig = plt.figure(1, (18,8))
```

```
for i, num_feature in enumerate(df_nums):
    ax = plt.subplot(2,3,i+1)
    sm.qqplot(transformed_df[num_feature], line='s', ax=ax, markersize=5)
    ax.set_title(f'qqplot - {num_feature}')
    plt.tight_layout()
plt.show()
```

Visualization after applying power transformation

Table Raw Visualize Statistics

	age	avg_glucose...	bmi
0	-2.07926422980449	-1.033592581228881	-1.91696012881604...
1	-2.07926422980449	1.0636591548902505	-2.594946743020776
2	-2.073509203929598	0.6079989337891626	-1.80721138514531...
3	-2.073509203929598	0.486324685327995	-2.648115135903927
4	-2.073509203929598	-1.066805499151809	-2.89660504355823...



FEATURE ENGINEERING

Steps of feature engineering

1. Univariate analysis
2. Bi-variate analysis
3. Chi-square test
 1. Stroke data has more of categorical variables
4. Correlation heat map

UNIVARIATE ANALYSIS

Change Binominal Categorical to Numerical

```
df['gender'].replace(['Male', 'Female'],
                    [0, 1], inplace=True)
df['ever_married'].replace(['No', 'Yes'],
                          [0, 1], inplace=True)
df['work_type'].replace(['Never_worked', 'Govt_job', 'children', 'Self-employed', 'Private'],
                       [0, 1, 2, 3, 4], inplace=True)
df['Residence_type'].replace(['Rural', 'Urban'],
                             [0, 1], inplace=True)
df['smoking_status'].replace(['smokes', 'formerly smoked', 'Unknown', 'never smoked'],
                             [0, 1, 2, 3], inplace=True)

print(df)
```

4094	1	111.81	19.8	1	0
2341	1	80.00	33.6	3	0
4716	0	96.98	21.5	3	0
187	1	215.94	27.9	1	1

	age_range	Traditional
3295	[0, 2)	1
1614	[0, 2)	1
3618	[0, 2)	1
4021	[0, 2)	1
3968	[0, 2)	1
...	...	...
4590	[20, 85)	1
4094	[20, 85)	1
2341	[20, 85)	1
4716	[20, 85)	1
187	[20, 85)	-1

[5110 rows x 13 columns]

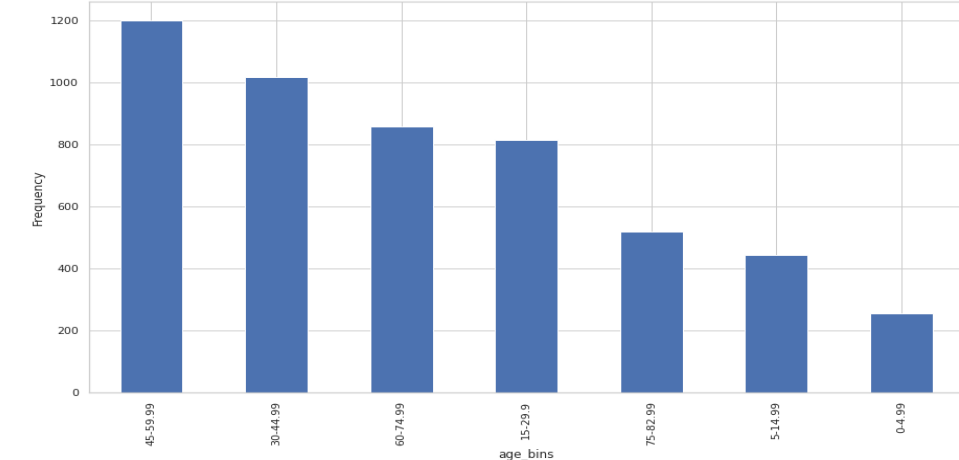
Age_bins

```
# Level1 FREQUENCY and PERCENTAGE of the unique age_bins values Asma, Y
import matplotlib.pyplot as plt
freq = df['age_bins'].value_counts(dropna=False)
percent=df['age_bins'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("age_bins")
plt.ylabel("Frequency")
```

	frequency	percent
45-59.99	1201	23.502935
30-44.99	1018	19.921722
60-74.99	858	16.790607
15-29.9	816	15.968689
75-82.99	518	10.136986
5-14.99	444	8.688845
0-4.99	255	4.990215

Text(0, 0.5, 'Frequency')



UNIVARIATE ANALYSIS

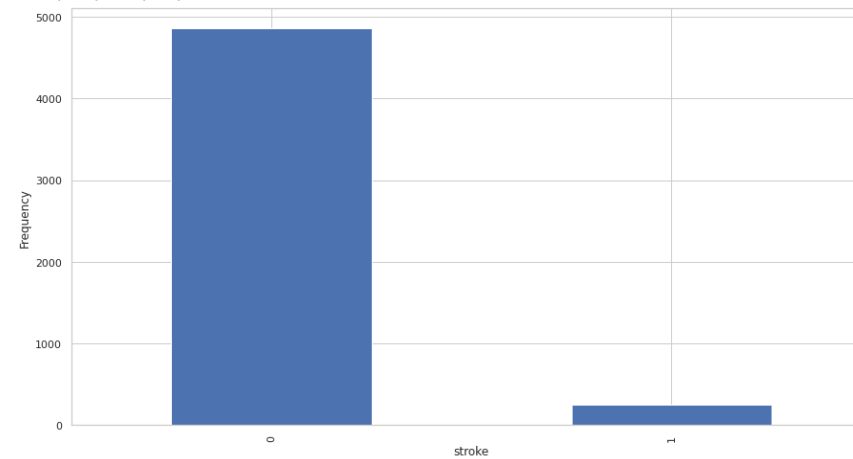
Stroke

```
# Level1 FREQUENCY and PERCENTAGE of the unique STROKE values Asma, Y
import matplotlib.pyplot as plt
freq = df['stroke'].value_counts(dropna=False)
percent=df['stroke'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("stroke")
plt.ylabel("Frequency")
```

	frequency	percent
0	4861	95.127282
1	249	4.872798

```
Text(0, 0.5, 'Frequency')
```



UNIVARIATE ANALYSIS

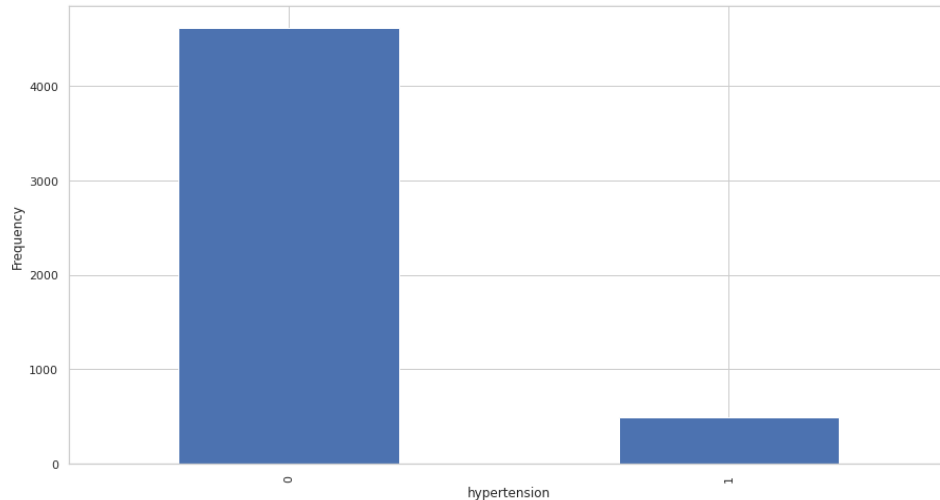
Hypertension

```
# Level1 FREQUENCY and PERCENTAGE of the unique hypertension values Asma, Y
import matplotlib.pyplot as plt
freq = df['hypertension'].value_counts(dropna=False)
percent=df['hypertension'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("hypertension")
plt.ylabel("Frequency")
```

	frequency	percent
0	4612	90.254403
1	498	9.745597

Text(0, 0.5, 'Frequency')



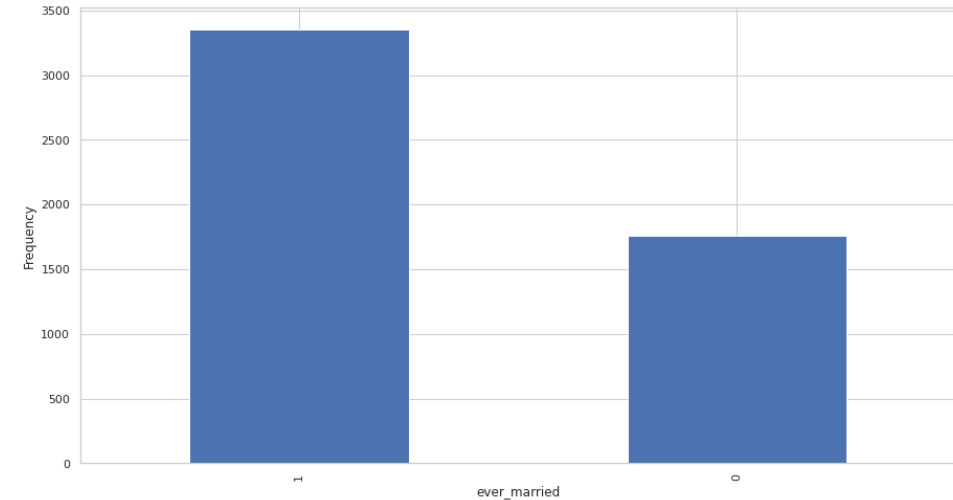
Ever_married

```
# Level1 FREQUENCY and PERCENTAGE of the unique hypertension values Asma, Y
import matplotlib.pyplot as plt
freq = df['ever_married'].value_counts(dropna=False)
percent=df['ever_married'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("ever_married")
plt.ylabel("Frequency")
```

	frequency	percent
1	3353	65.616438
0	1757	34.383562

Text(0, 0.5, 'Frequency')



UNIVARIATE ANALYSIS

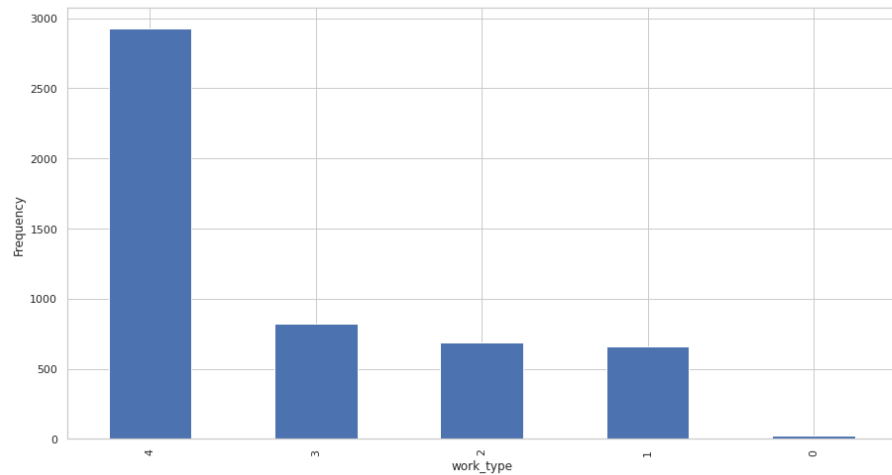
Work_type

```
# Level1 FREQUENCY and PERCENTAGE of the unique work_type values Asma, Y
import matplotlib.pyplot as plt
freq = df['work_type'].value_counts(dropna=False)
percent=df['work_type'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("work_type")
plt.ylabel("Frequency")
```

	frequency	percent
4	2925	57.240705
3	819	16.027397
2	687	13.444227
1	657	12.857143
0	22	0.430528

Text(0, 0.5, 'Frequency')



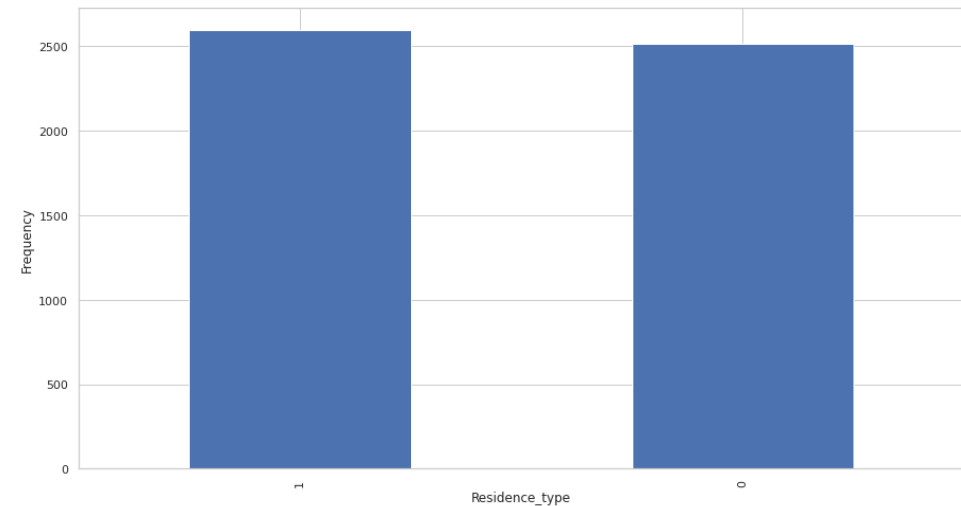
Residence_type

```
# Level1 FREQUENCY and PERCENTAGE of the unique Residence_type values Asma, Y
import matplotlib.pyplot as plt
freq = df['Residence_type'].value_counts(dropna=False)
percent=df['Residence_type'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("Residence_type")
plt.ylabel("Frequency")
```

	frequency	percent
1	2596	50.802348
0	2514	49.197652

Text(0, 0.5, 'Frequency')



UNIVARIATE ANALYSIS

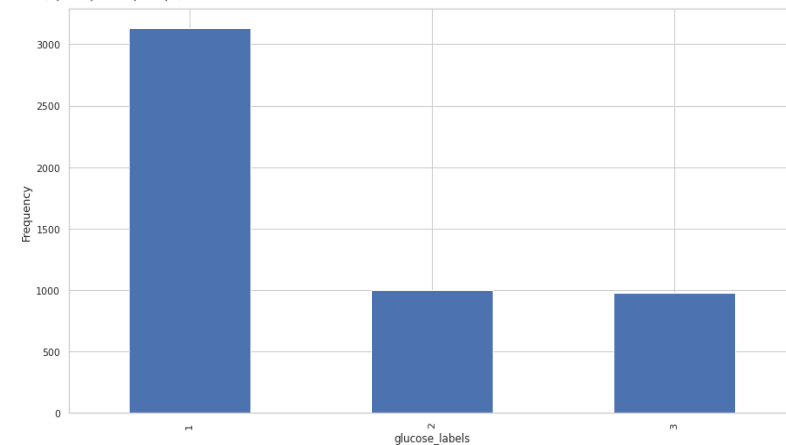
Glucose_labels

```
# Level1 FREQUENCY and PERCENTAGE of the unique glucose_labels values Asma, Y
import matplotlib.pyplot as plt
freq = df['glucose_labels'].value_counts(dropna=False)
percent=df['glucose_labels'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax, kind='bar')
plt.xlabel("glucose_labels")
plt.ylabel("Frequency")
```

	frequency	percent
1	3131	61.272016
2	998	19.530333
3	981	19.197652

Text(0, 0.5, 'Frequency')



Glucose_labels . Min & Max

```
#LAYER 1 (Yuliya)
# Finding the max ans min for binning
min_value = df['avg_glucose_level'].min()
max_value = df['avg_glucose_level'].max()
print(min_value)
print(max_value)
```

55.12
271.74

```
#LAYER 1 (Yuliya)
'''BMI WEIGHT STATUS Binning
Below 18.5 Underweight 1
18.5 - 24.9 Normal 2
25.0 - 29.9 Overweight 3
30.0 and above Obesity 4
The left bin edge will be exclusive and the right bin edge will be inclus
#LAYER 1 (Yuliya)
# Finding the max ans min for binning
min_value = df['bmi'].min()
max_value = df['bmi'].max()
print(min_value)
print(max_value)
#BMI BINNING 1:10.30-18.49, 2:18.5-24.99, 3:25-29.99 4:30-max
bmi_labels = [1, 2, 3, 4]
cut_bins = [10.20, 18.49, 24.99, 29.99, df["bmi"].max()]
df['bmi_labels'] = pd.cut(df['bmi'], bins=cut_bins, labels=bmi_labels)
```

10.3
97.6

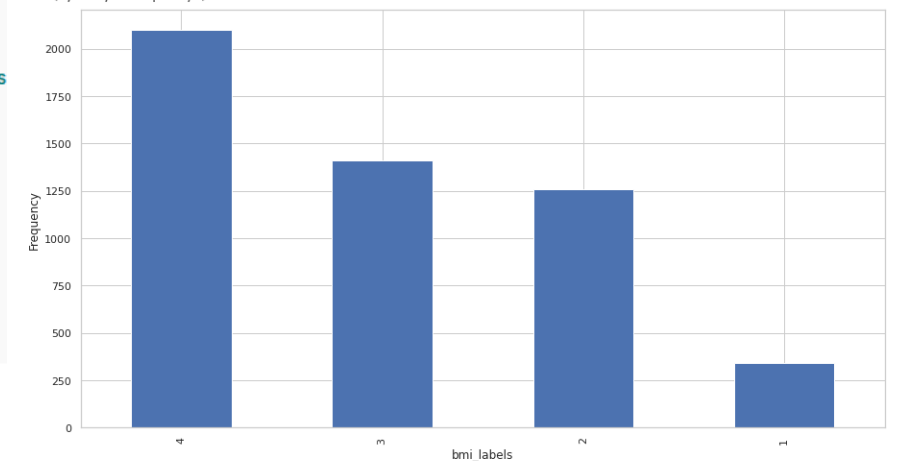
BMI_labels

```
# Level1 FREQUENCY and PERCENTAGE of the unique bmi_labels values Asma, Y
import matplotlib.pyplot as plt
freq = df['bmi_labels'].value_counts(dropna=False)
percent=df['bmi_labels'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax, kind='bar')
plt.xlabel("bmi_labels")
plt.ylabel("Frequency")
```

	frequency	percent
4	2099	41.076321
3	1409	27.573386
2	1259	24.637965
1	343	6.712329

Text(0, 0.5, 'Frequency')



UNIVARIATE ANALYSIS

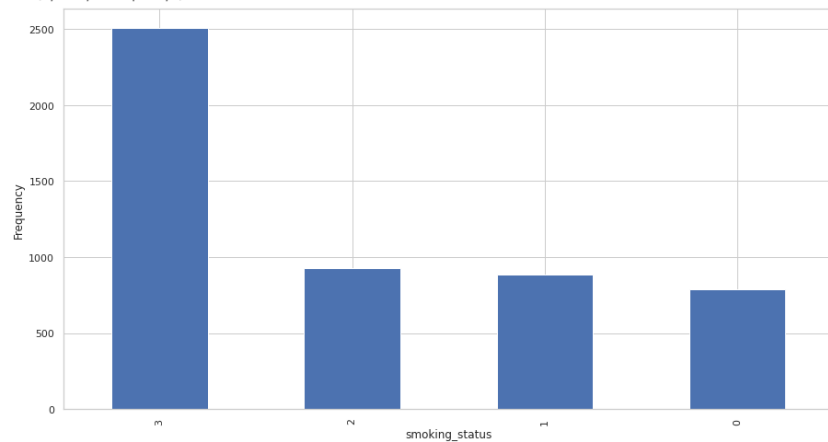
Smoking_status

```
# Level1 FREQUENCY and PERCENTAGE of the unique smoking_status values Asma, Y
import matplotlib.pyplot as plt
freq = df['smoking_status'].value_counts(dropna=False)
percent=df['smoking_status'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("smoking_status")
plt.ylabel("Frequency")
```

	frequency	percent
3	2506	49.841096
2	930	18.199609
1	885	17.318982
0	789	15.440313

Text(0, 0.5, 'Frequency')



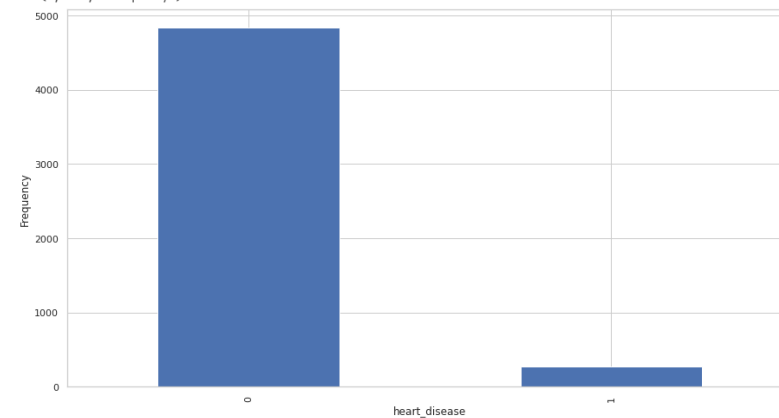
Heart_disease

```
# Level1 FREQUENCY and PERCENTAGE of the unique HEART_DISEASE values Asma, Y
import matplotlib.pyplot as plt
freq = df['heart_disease'].value_counts(dropna=False)
percent=df['heart_disease'].value_counts(normalize=True)*100
new_df=pd.DataFrame({"frequency":freq,"percent":percent})
print(new_df)
```

```
fig, ax = plt.subplots()
freq.plot(ax=ax,kind='bar')
plt.xlabel("heart_disease")
plt.ylabel("Frequency")
```

	frequency	percent
0	4834	94.598826
1	276	5.401174

Text(0, 0.5, 'Frequency')



BIVARIATE ANALYSIS

Stroke vs Gender

```
##Layer 2 Cross Tab Stroke vs Gender (C)  
pd.crosstab(df.stroke,df.gender,margins=True)
```

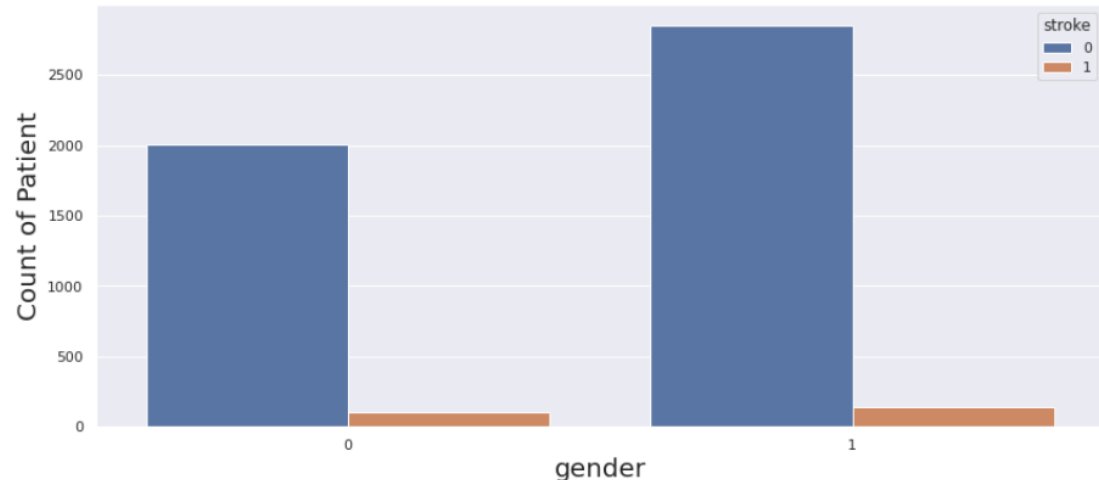
gender	0	1	All
stroke			
0	2007	2854	4861
1	108	141	249
All	2115	2995	5110

```
#Layer 2 Percentage Cross Tab Stroke vs gender (C)  
pd.crosstab(df.stroke,df.gender, normalize='index')
```

gender	0	1
stroke		
0	0.412878	0.587122
1	0.433735	0.566265

There may be a certain correlation between the probability of stroke and gender.

```
#Layer 2 Plot gender (C)  
print(df['gender'].value_counts())  
sns.set(rc={'figure.figsize':(14,6)})  
seaborn_plot = sns.countplot(df['gender'], hue = df['stroke'])  
seaborn_plot.set_xlabel("gender",fontsize=20)  
seaborn_plot.set_ylabel("Count of Patient",fontsize=20)  
# the # of female gender is more than man for the dataset so this variable itself is no
```



BIVARIATE ANALYSIS

Stroke vs Hypertension

```
#Layer 2 Cross Tab Stroke vs hypertension (C)  
pd.crosstab(df.stroke,df.hypertension,margins=True)
```

hypertension	0	1	All
stroke			
0	4429	432	4861
1	183	66	249
All	4612	498	5110

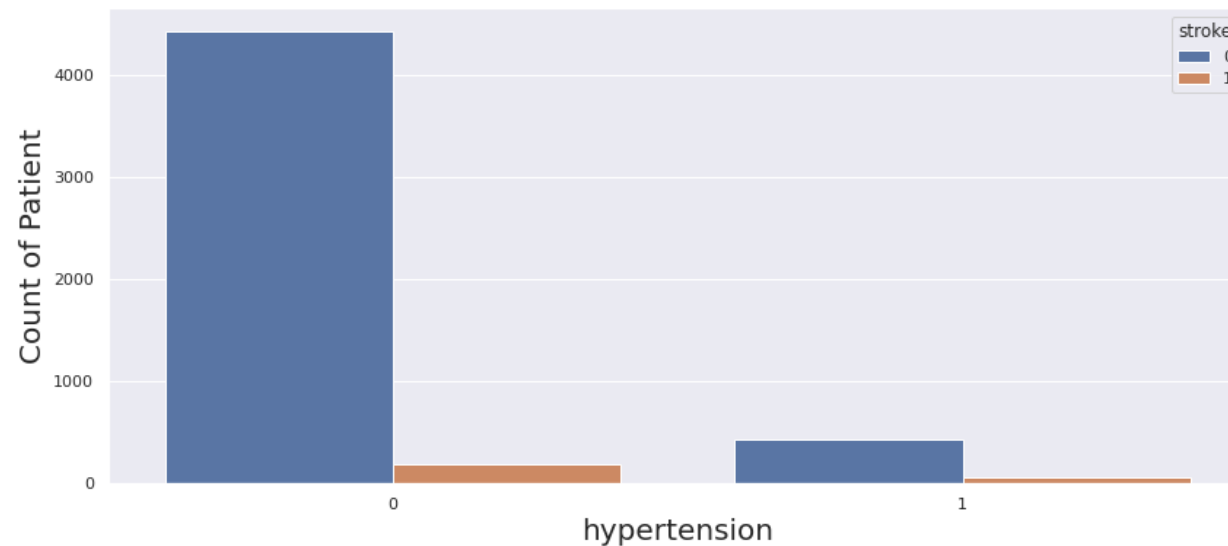
```
#Layer 2 Percentage Cross Tab Stroke vs hypertension (C)  
pd.crosstab(df.stroke,dataset.hypertension, normalize='index')
```

hypertension	0	1
stroke		
0	0.911129	0.088871
1	0.734940	0.265060

```
#Layer 2 Plot hypertension (C)  
print(df['hypertension'].value_counts())  
sns.set(rc={'figure.figsize':(14,6)})  
seaborn_plot = sns.countplot(df['hypertension'], hue = df['stroke'])  
seaborn_plot.set_xlabel("hypertension",fontsize=20)  
seaborn_plot.set_ylabel("Count of Patient",fontsize=20)
```

```
0    4612  
1     498  
Name: hypertension, dtype: int64
```

```
Text(0, 0.5, 'Count of Patient')
```



BIVARIATE ANALYSIS

Stroke vs Heart_disease

```
#Layer 2 Cross Tab Stroke vs heart_disease (C)
pd.crosstab(df.stroke,df.heart_disease,margins=True)
```

heart_disease	0	1	All
stroke			
0	4632	229	4861
1	202	47	249
All	4834	276	5110

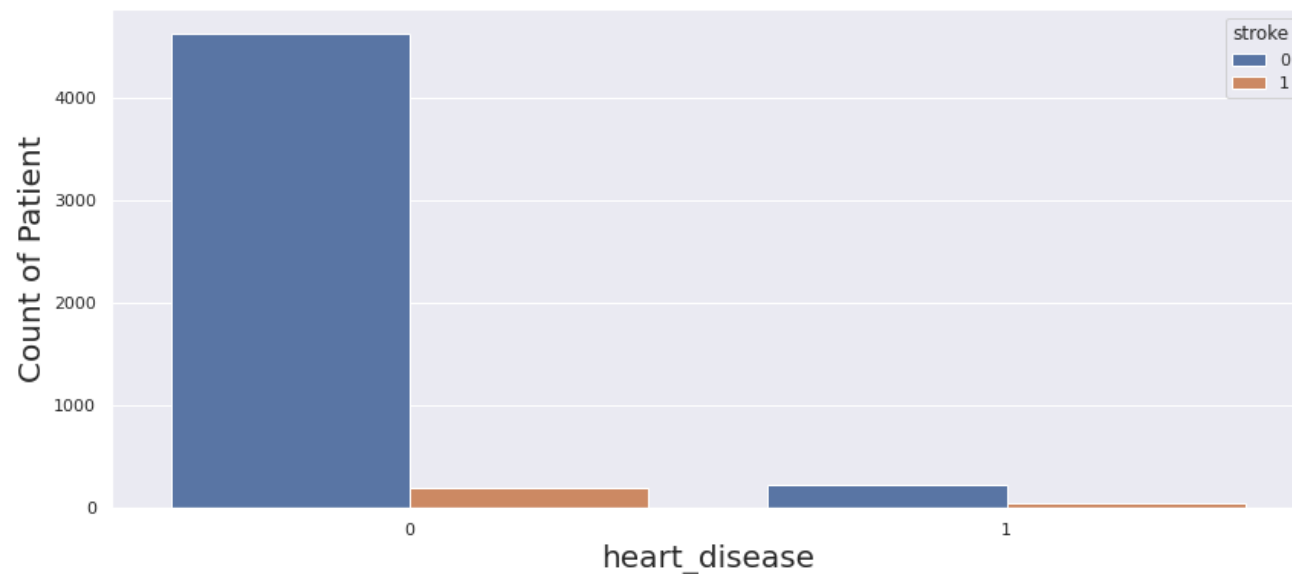
```
#Layer 2 Percentage Cross Tab Stroke vs heart_disease (C)
pd.crosstab(df.stroke,df.heart_disease, normalize='index')
```

heart_disease	0	1
stroke		
0	0.952890	0.047110
1	0.811245	0.188755

```
#Layer 2 Plot heart_disease (C)
print(df['heart_disease'].value_counts())
sns.set(rc={'figure.figsize':(14,6)})
seaborn_plot = sns.countplot(df['heart_disease'], hue = df['stroke'])
seaborn_plot.set_xlabel("heart_disease",fontsize=20)
seaborn_plot.set_ylabel("Count of Patient",fontsize=20)
```

```
0    4834
1     276
Name: heart_disease, dtype: int64
```

```
Text(0, 0.5, 'Count of Patient')
```



BIVARIATE ANALYSIS

Stroke vs Ever_married

```
#Layer 2 Cross Tab Stroke vs ever_married (C)
pd.crosstab(df.stroke,dataset.ever_married,margins=True)
```

ever_married	No	Yes	All
stroke			
0	1728	3133	4861
1	29	220	249
All	1757	3353	5110

```
#Layer 2 Percentage Cross Tab Stroke vs ever_married (C)
pd.crosstab(df.stroke,df.ever_married, normalize='index')
```

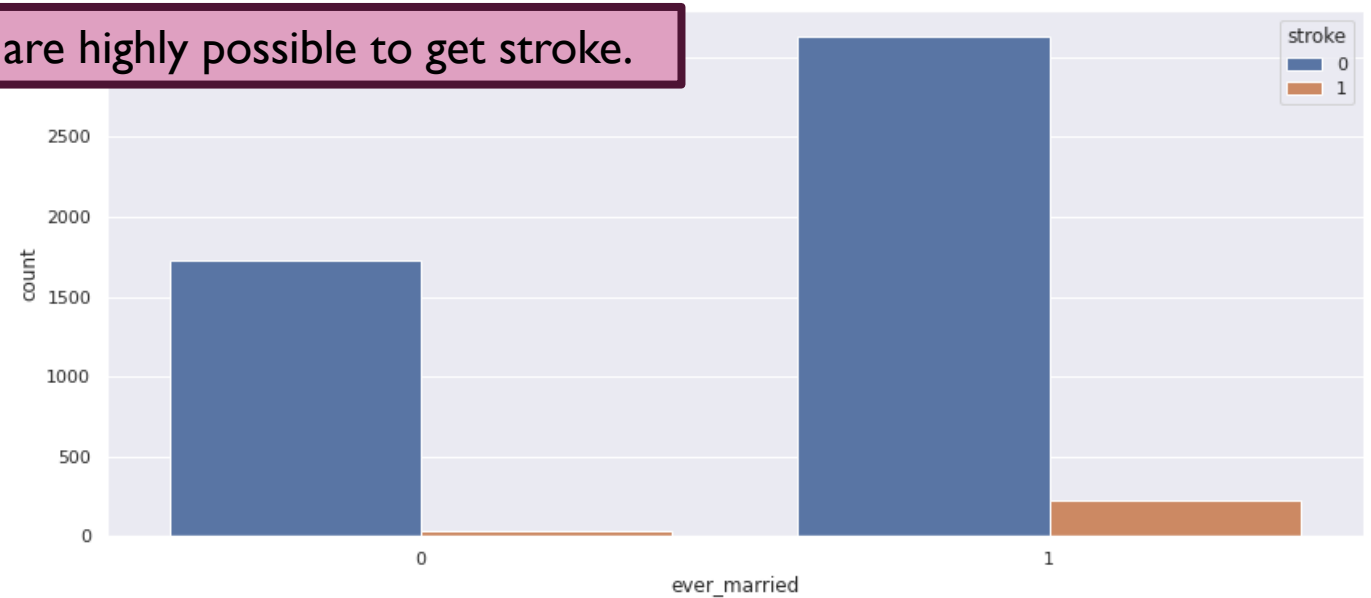
ever_married	0	1
stroke		
0	0.355482	0.644518
1	0.116466	0.883534

```
#Layer 2 Plot ever_married (C)
print(df['ever_married'].value_counts())
sns.countplot(df['ever_married'],hue = df['stroke'])
```

```
1    3353
0    1757
Name: ever_married, dtype: int64
```

```
<AxesSubplot:xlabel='ever_married', ylabel='count'>
```

Married people are highly possible to get stroke.



BIVARIATE ANALYSIS

Stroke vs Work_type

```
#Layer 2 Cross Tab Stroke vs work_type (C)
pd.crosstab(df.stroke,df.work_type,margins=True)
```

work_type	0	1	2	3	4	All
stroke						
0	22	624	685	754	2776	4861
1	0	33	2	65	149	249
All	22	657	687	819	2925	5110

```
#Layer 2 Percentag Cross Tab Stroke vs work_type (C)
pd.crosstab(df.stroke,df.work_type, normalize='index')
```

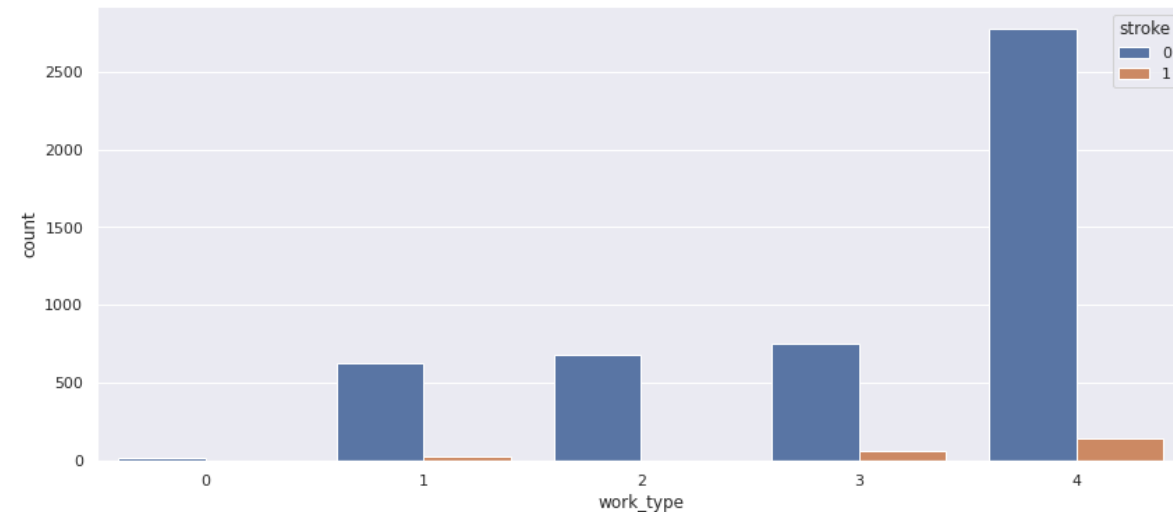
work_type	0	1	2	3	4
stroke					
0	0.004526	0.128369	0.140918	0.155112	0.571076
1	0.000000	0.132530	0.008032	0.261044	0.598394

```
#Layer 2 (C)
print(df['work_type'].value_counts())
sns.countplot(df['work_type'], hue = df['stroke'])
```

*#We don't have much data on patients that never worked but the data we have in that category never suffered from a stroke. Also the i
Its obvious that rarely children have stroke. The work type variable by itself does not tell much about the stroke*

```
4    2925
3     819
2     687
1     657
0        22
Name: work_type, dtype: int64
```

<AxesSubplot:xlabel='work_type', ylabel='count'>



BIVARIATE ANALYSIS

Stroke vs Residence_type

```
#Layer 2 Cross Tab Stroke vs Residence_type (C)  
pd.crosstab(df.stroke,df.Residence_type ,margins=True)
```

Residence_type	0	1	All
stroke			
0	2400	2461	4861
1	114	135	249
All	2514	2596	5110

```
#Layer 2 Percentage Cross Tab Stroke vs Residence_type (C)  
pd.crosstab(df.stroke,df.Residence_type, normalize='index')
```

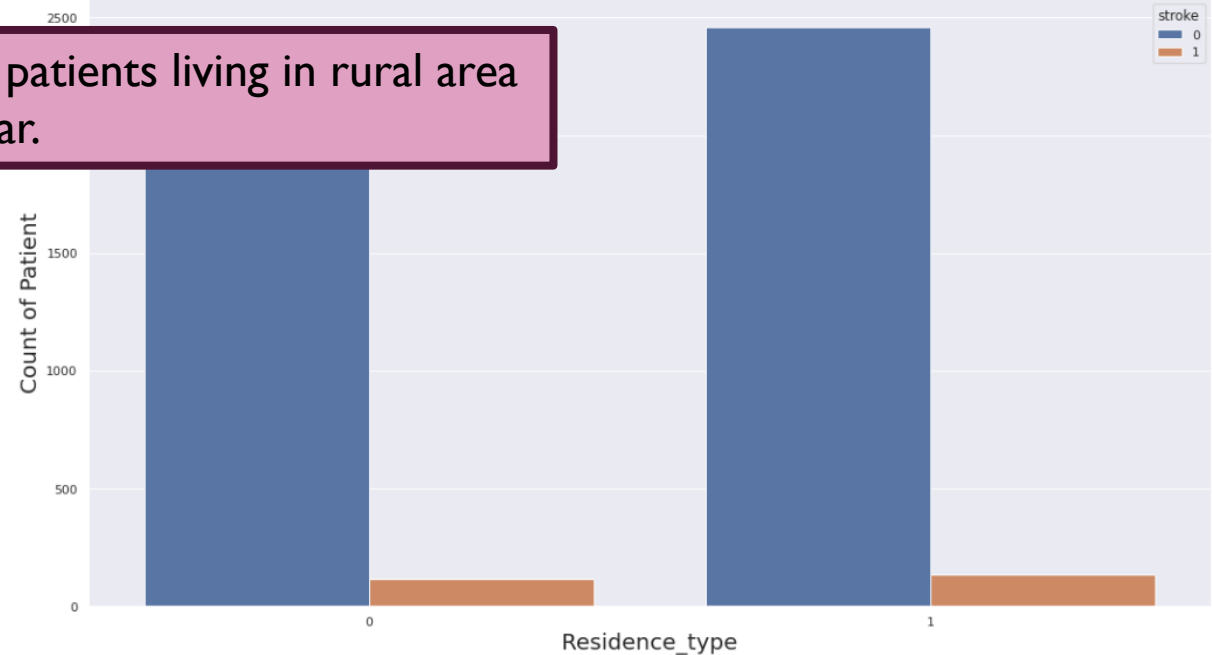
Residence_type	0	1
stroke		
0	0.493726	0.506274
1	0.457831	0.542169

The number of stroke patients living in rural area and urban area is similar.

```
#Layer 2 (C)  
print(df['Residence_type'].value_counts())  
sns.set(rc={'figure.figsize':(18,10)})  
seaborn_plot = sns.countplot(df['Residence_type'], hue = df['stroke'])  
seaborn_plot.set_xlabel("Residence_type",fontsize=20)  
seaborn_plot.set_ylabel("Count of Patient",fontsize=20)
```

```
1    2596  
0    2514  
Name: Residence_type, dtype: int64
```

```
Text(0, 0.5, 'Count of Patient')
```



BIVARIATE ANALYSIS

Stroke vs Smoking_status

```
#Layer 2 Cross Tab Stroke vs smoking_status (C)
pd.crosstab(df.stroke,df.smoking_status,margins=True)
```

smoking_status	0	1	2	3	All
stroke					
0	747	815	885	2414	4861
1	42	70	45	92	249
All	789	885	930	2506	5110

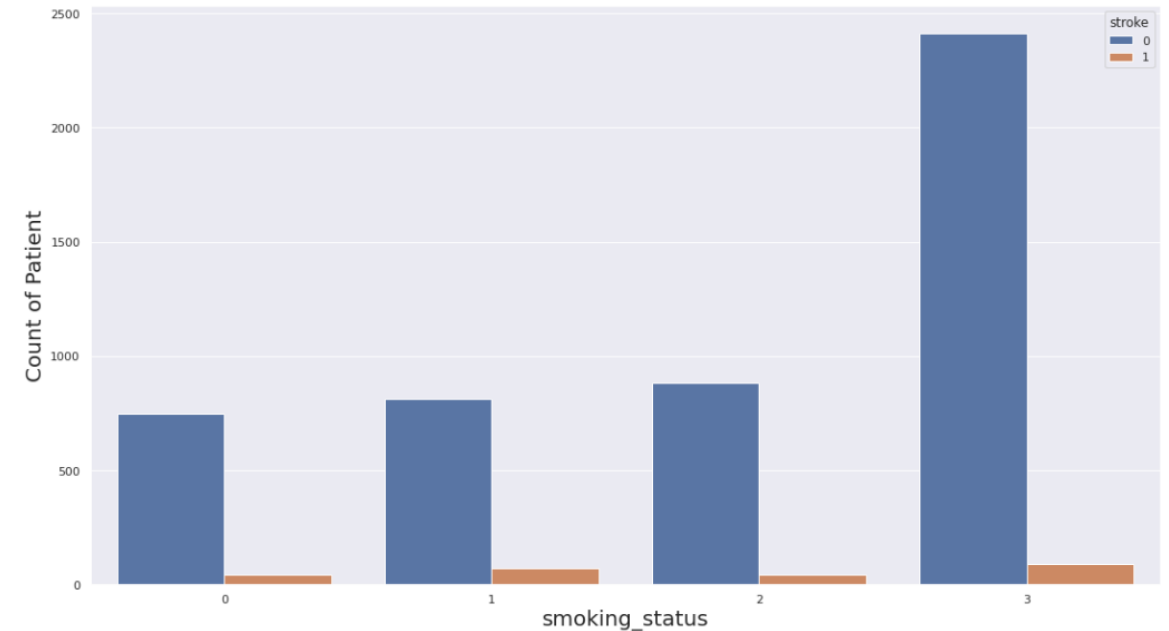
```
##Layer 2 Percentage Cross Tab Stroke vs smoking status (C)
pd.crosstab(df.stroke,df.smoking_status, normalize='index')
```

smoking_status	0	1	2	3
stroke				
0	0.153672	0.167661	0.182061	0.496606
1	0.168675	0.281124	0.180723	0.369478

```
#Layer 2 Plot smoking_status (C)
print(df['smoking_status'].value_counts())
sns.set(rc={'figure.figsize':(18,10)})
seaborn_plot = sns.countplot(df['smoking_status'], hue = df['stroke'])
seaborn_plot.set_xlabel("smoking_status",fontsize=20)
seaborn_plot.set_ylabel("Count of Patient",fontsize=20)
```

```
3    2506
2     930
1     885
0     789
Name: smoking_status, dtype: int64
```

Text(0, 0.5, 'Count of Patient')



BIVARIATE ANALYSIS

Stroke vs Age_bins

```
#Layer 2 Cross tab Stroke-Age bins (Y)
pd.crosstab(df.stroke, df.age_bins, margins=True)
```

age_bins	0-4.99	5-14.99	15-29.9	30-44.99	45-59.99	60-74.99	75-82.99	All
stroke								
0	254	443	816	1010	1143	779	416	4861
1	1	1	0	8	58	79	102	249
All	255	444	816	1018	1201	858	518	5110

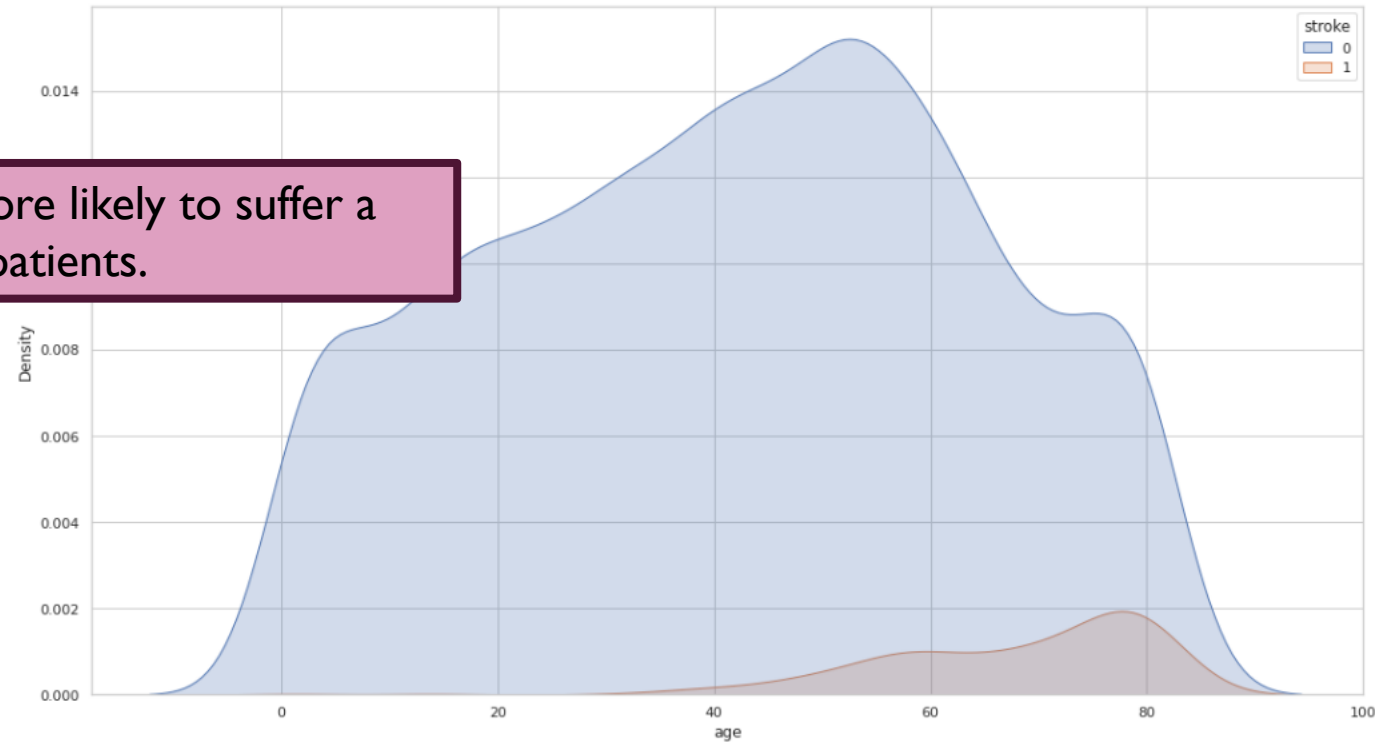
```
#Layer 2 Percentage of Cross Tabulation Stroke vs Age_bins(Y)
all=pd.crosstab(df.stroke, df.age_bins, margins=True)["All"] #deviding by all
pd.crosstab(df.stroke, df.age_bins).divide(all,axis=0).dropna()
```

age_bins	0-4.99	5-14.99	15-29.9	30-44.99	45-59.99	60-74.99	75-82.99
stroke							
0	0.052253	0.091134	0.167867	0.207776	0.235137	0.160255	0.085579
1	0.004016	0.004016	0.000000	0.032129	0.232932	0.317269	0.409639

Older patients are more likely to suffer a stroke than younger patients.

```
# Kernel Distribution Estimation Plot which depicts the probability density function of the continuous (Y)
#or non-parametric data variables
sns.kdeplot(data=df, x="age", hue="stroke", fill=True)
```

<AxesSubplot:xlabel='age', ylabel='Density'>



BIVARIATE ANALYSIS

Stroke vs Avg_glucose_level

```
#Layer 2 Cross tab Stroke vs glucose_labels (Y)
pd.crosstab(df.stroke, df.glucose_labels, margins=True)
```

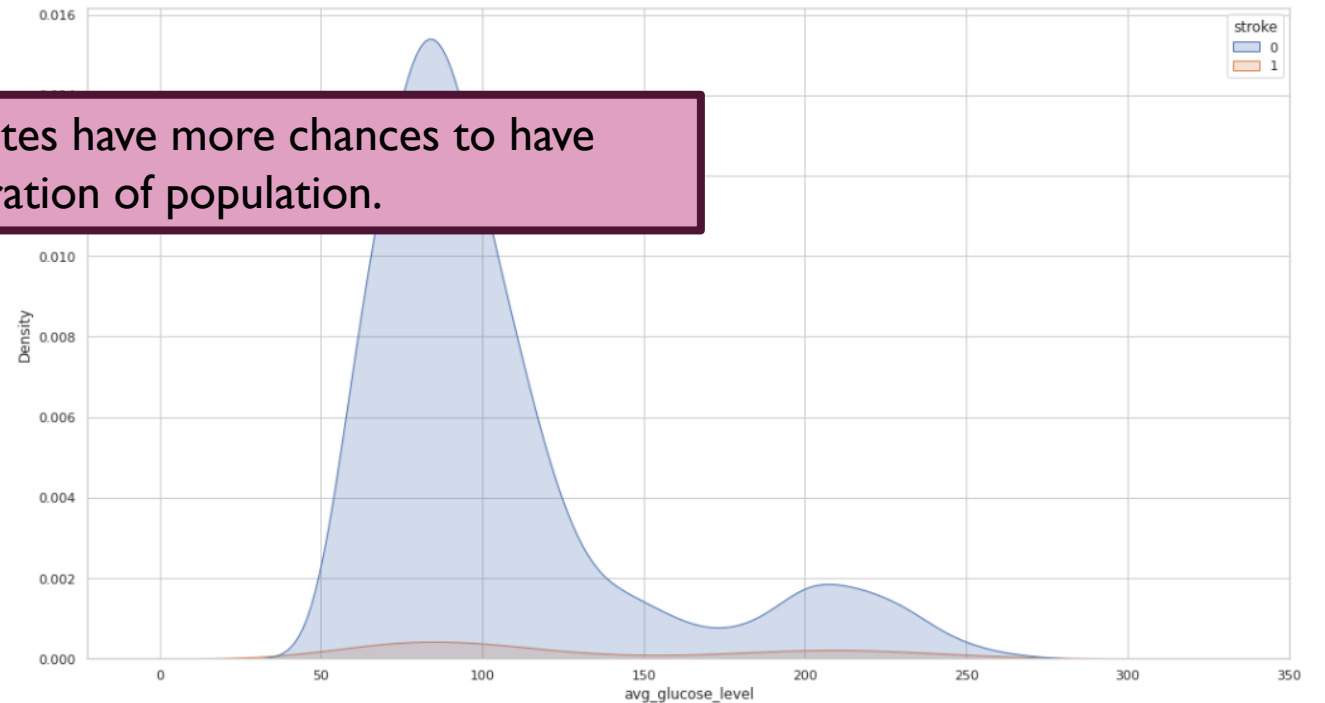
glucose_labels	1	2	3	All
stroke				
0	3019	961	881	4861
1	112	37	100	249
All	3131	998	981	5110

```
#Layer 2 Percentage of Cross Tabulation Stroke vs glucose_labels(Y)
all=pd.crosstab(df.stroke, df.glucose_labels, margins=True)["All"] #deviding by all
pd.crosstab(df.stroke, df.glucose_labels).divide(all, axis=0).dropna()
```

glucose_labels	1	2	3
stroke			
0	0.621066	0.197696	0.181238
1	0.449799	0.148594	0.401606

```
# Kernel Distribution Estimation Plot (Y)
sns.kdeplot(data=df, x="avg_glucose_level", hue="stroke", fill=True)
```

<AxesSubplot:xlabel='avg_glucose_level', ylabel='Density'>



The patients with diabetes have more chances to have stroke in terms of the ration of population.

BIVARIATE ANALYSIS

Stroke vs BMI

```
#Layer 2 Cross tab Stroke vs bmi_labels (Y)
pd.crosstab(df.stroke, df.bmi_labels, margins=True)
```

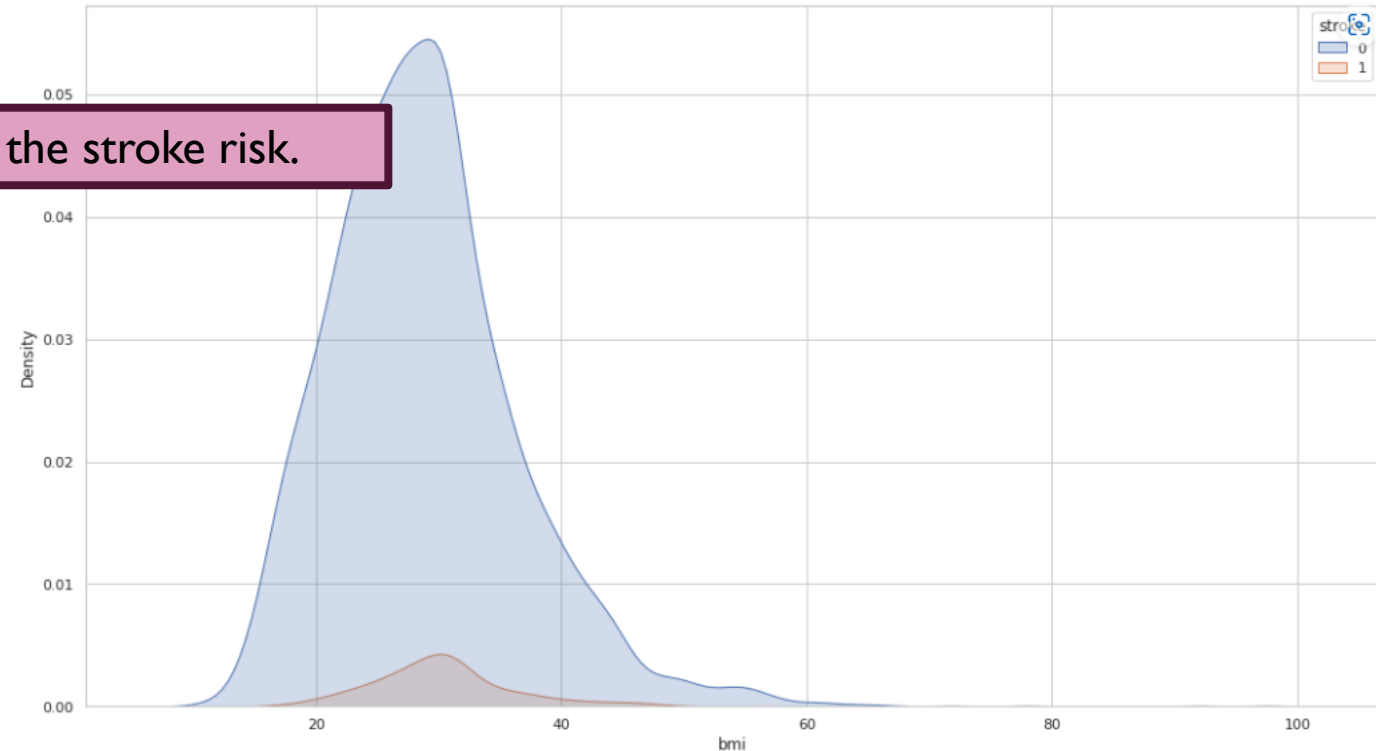
bmi_labels	1	2	3	4	All
stroke					
0	341	1224	1334	1962	4861
1	2	35	75	137	249
All	343	1259	1409	2099	5110

```
#Layer 2 Percentage of Cross Tabulation Stroke vs bmi_labels(Y)
all=pd.crosstab(df.stroke, df.bmi_labels, margins=True)["All"] #dividing by a
pd.crosstab(df.stroke, df.bmi_labels).divide(all, axis=0).dropna()
```

bmi_labels	1	2	3	4
stroke				
0	0.070150	0.251800	0.274429	0.403621
1	0.008032	0.140562	0.301205	0.550201

```
# Kernel Distribution Estimation Plotb (Y)
sns.kdeplot(data=df, x="bmi", hue="stroke", fill=True)
```

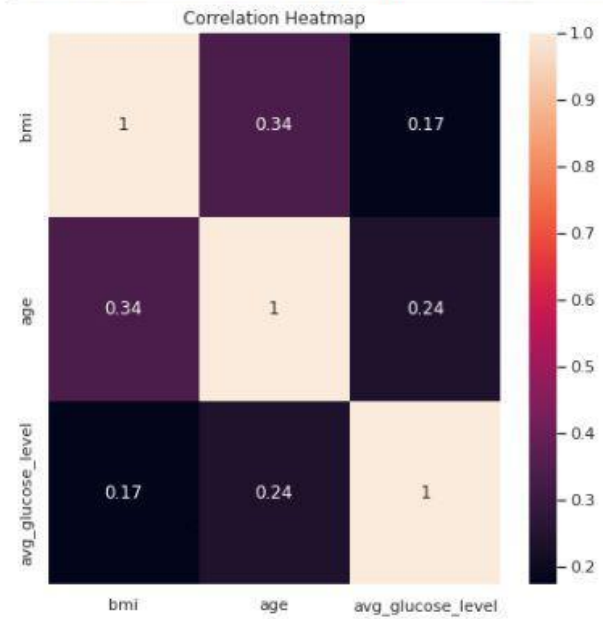
<AxesSubplot:xlabel='bmi', ylabel='Density'>



CORRELATION

```
0.6s
#Basic Seaborn Heatmap for 2 continuous variables
# The Pearson correlation coefficient can range from -1 to 1
fig, ax = plt.subplots(figsize=(7, 7))
heatmap = sns.heatmap(df[["bmi", "age", "avg_glucose_level"]].corr(), vmax=1, annot=True, ax = ax)
heatmap.set_title('Correlation Heatmap')
'''Correlation 0.9 and 1.0 very highly correlated.
Correlation 0.7 and 0.9 highly correlated.
Correlation 0.5 and 0.7 moderately correlated.
Correlation 0.3 and 0.5 low correlation.
Correlation less than 0.3 have little if any (linear) correlation.'''

'Correlation 0.9 and 1.0 very highly correlated. \nCorrelation 0.7 and 0.9 highly correlated. \nCo
```



T-TEST

```
#T-test Gender and bmi. The null hypothesis (H0) is that the true difference between these group means is zero.
#Distribution gender grouped by avg_glucose_level ,where female is 1 , male 0 (cony. variables)
ALPHA=0.05
x, y = df[df.gender==0].bmi, df[df.gender==1].bmi

t, p = stats.ttest_ind(x, y, equal_var=False)
t, p = round(t, 5), round(p, 5)

if p <= ALPHA:
    print(f'Since p = {p} < {ALPHA}, reject H0, different bmi distributions between 0 and 1 (t={t}, p={p})')
else:
    print(f'Since p = {p} > {ALPHA}, fail to reject H0, same bmi distributions between 0 and 1 (t={t}, p={p}).')
```

Since $p = 0.07043 > 0.05$, fail to reject H_0 , same bmi distributions between 0 and 1 ($t=-1.80951$, $p=0.07043$).

T-TEST

```
#T-test Gender and Age. The null hypothesis (H0) is that the true difference between these group means is zero.
#Distribution gender grouped by avg_glucose_level ,where female is 1 , male 0 (cony. variables)
ALPHA = 0.05
from scipy import stats
x, y = df[df.gender==0].age, df[df.gender==1].age

t, p = stats.ttest_ind(x, y, equal_var=False)
t, p = round(t, 5), round(p, 5)

if p <= ALPHA:
    print(f'Since p = {p} < {ALPHA}, reject H0, different Age distributions between 0 and 1 (t={t}, p={p})')
else:
    print(f'Since p = {p} > {ALPHA}, fail to reject H0, same Age distributions between 0 and 1 (t={t}, p={p}).')
```

Since $p = 0.05096 > 0.05$, fail to reject H_0 , same Age distributions between 0 and 1 ($t=-1.95239$, $p=0.05096$).

```
▶ 0.0s
#T-test Gender and Average glucose level. The null hypothesis (H0) is that the true difference between these group means is zero.
#Distribution gender grouped by avg_glucose_level ,where female is 1 , male 0 (cony. variables)
ALPHA=0.05
x, y = df[df.gender==0].avg_glucose_level, df[df.gender==1].avg_glucose_level

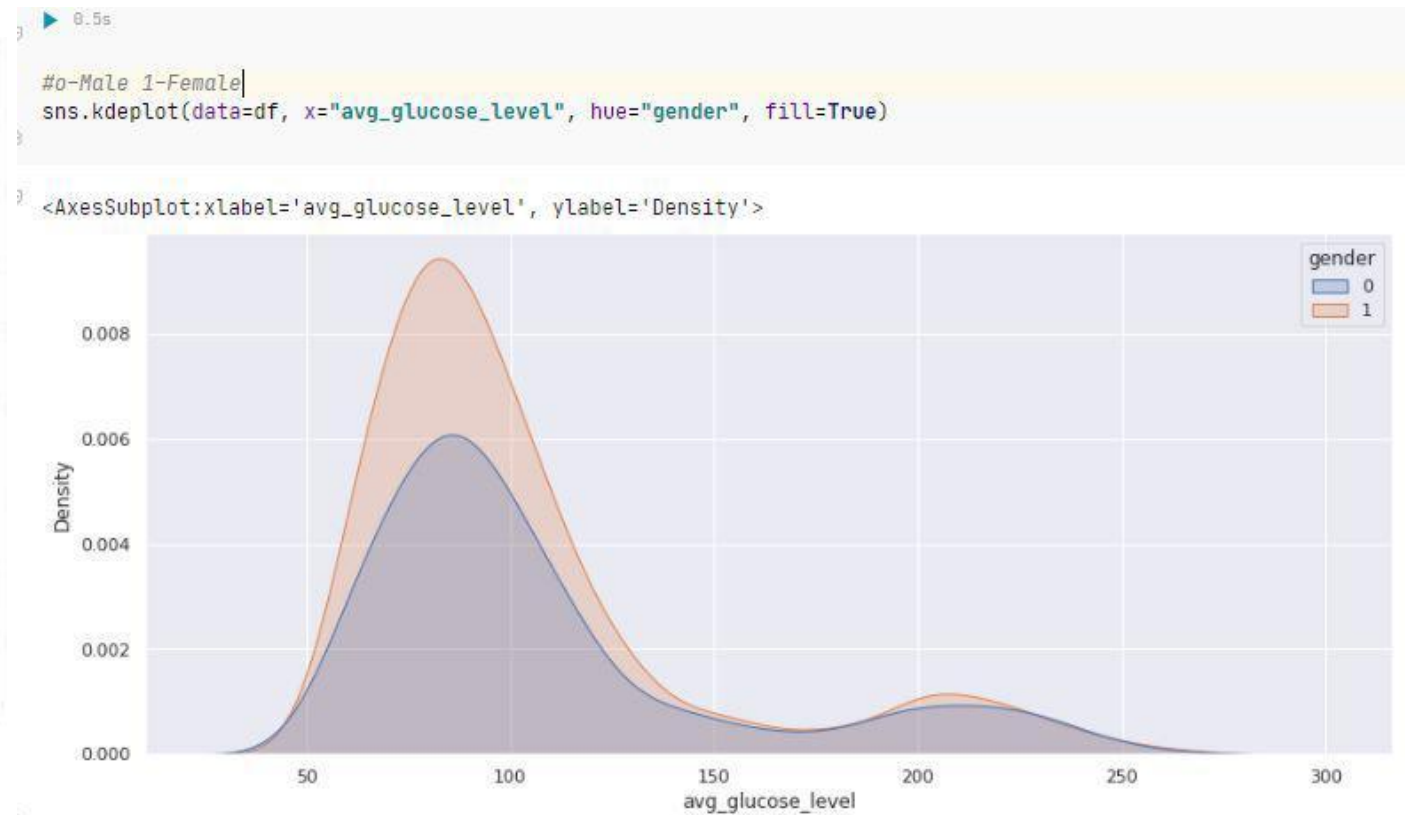
t, p = stats.ttest_ind(x, y, equal_var=False)
t, p = round(t, 5), round(p, 5)

if p <= ALPHA:
    print(f'Since p = {p} < {ALPHA}, reject H0, different avg_glucose_level distributions between 0 and 1 (t={t}, p={p})')
else:
    print(f'Since p = {p} > {ALPHA}, fail to reject H0, same avg_glucose_level distributions between 0 and 1 (t={t}, p={p}).')
```

Since $p = 0.00012 < 0.05$, reject H_0 , different avg_glucose_level distributions between 0 and 1 ($t=3.85029$, $p=0.00012$)

T-TEST

	0	1	2
#	0	1	1
cat_feat	gender	gender	gender
num_feat	age	avg_glucose_level	bmi
alpha	0.05	0.05	0.05
t	-1.95	-6.9	-1.8
p-value	0.05	0.00	0.07
H0	fail to reject	rejected	failed to reject
distribution	the same	different	the same



CHI-SQUARE TEST

Stroke Vs Work_type

```
#CHI-SQUARE STROKE and WORK TYPE
myF1= df['stroke']
myF2= df['work_type']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
chiVal, pVal, df1, exp
```

stroke	0	1
work_type		
0	22	0
1	624	33
2	685	2
3	754	65
4	2776	149

```
(49.16351197667529,
5.397707801896138e-10,
4,
array([[2.09279843e+01, 1.07201566e+00],
       [6.24985714e+02, 3.20142857e+01],
       [6.53523875e+02, 3.34761252e+01],
       [7.79091781e+02, 3.99082192e+01],
       [2.78247065e+03, 1.42529354e+02]]))
```

Chi-square(49.16), p-value (5.39e-10), degrees of freedom (4), expected frequencies are less than 5=> we can not full trust this test, reject $H_0 \Rightarrow$ there is a relationship.

Stroke Vs Age_bins

```
#CHI-SQUARE STROKE and AGE_BINS
myF1= df['stroke']
myF2= df['age_bins']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
chiVal, pVal, df1, exp
```

stroke	0	1
age_bins		
0-4.99	254	1
5-14.99	443	1
15-29.9	816	0
30-44.99	1010	8
45-59.99	1143	58
60-74.99	779	79
75-82.99	416	102

```
(390.3822065889208,
3.265283011373326e-81,
6,
array([[ 242.57436399, 12.42563601],
       [ 422.36477495, 21.63522505],
       [ 776.23796477, 39.76203523],
       [ 968.39491194, 49.60508806],
       [1142.4776908 , 58.5223092 ],
       [ 816.19138943, 41.80861057],
       [ 492.75890411, 25.24109589]]))
```

Chi-square(390.382), p-value (3.26e-81), degrees of freedom (6), expected frequencies are greater than 5=>we can trust results, reject $H_0 \Rightarrow$ there is a relationship.

CHI-SQUARE TEST

Stroke Vs Hypertension

```
myF1= df['stroke']
myF2= df['hypertension']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
chiVal, pVal, df1, exp
```

stroke	0	1
hypertension		
0	4429	183
1	432	66

```
(81.6053682482931,
 1.661621901511823e-19,
 1,
 array([[4387.2665362, 224.7334638],
        [ 473.7334638,  24.2665362]]))
```

Chi-square(81.6), p-value ($1.66e-19$), degrees of freedom (1), expected frequencies are greater than 5=> we can trust results, reject H_0 =>there is a relationship.

Stroke Vs Heart_disease

```
myF1= df['stroke']
myF2= df['heart_disease']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
chiVal, pVal, df1, exp
```

stroke	0	1
heart_disease		
0	4632	202
1	229	47

```
(90.25956125843324,
 2.0887845685229236e-21,
 1,
 array([[4598.44892368, 235.55107632],
        [ 262.55107632,  13.44892368]]))
```

Chi-square(90.6), p-value ($0.088e-21$), degrees of freedom (1), expected frequencies are greater than 5=>we can trust results, reject H_0 =>there is a relationship.

CHI-SQUARE TEST

Stroke Vs Age Ever_married

```
myF1= df['stroke']
myF2= df['ever_married']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
chiVal, pVal, df1, exp
```

stroke	0	1
ever_married		
0	1728	29
1	3133	220

```
(58.923890259034195,
1.6389021142314745e-14,
1,
array([[1671.38493151, 85.61506849],
       [3189.61506849, 163.38493151]]))
```

Chi-square(58.92), p-value (1.6e-14), degrees of freedom (1), expected frequencies are greater than 5=> we can trust results, reject H0=>there is a relationship.

Stroke Vs Residence_type

```
myF1= df['stroke']
myF2= df['Residence_type']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_
chiVal, pVal, df1, exp
```

stroke	0	1
Residence_type		
0	2400	114
1	2461	135

```
(1.0816367471627524,
0.29833169286876987,
1,
array([[2391.49784736, 122.50215264],
       [2469.50215264, 126.49784736]]))
```

Chi-square(1.08), p-value (0.29), degrees of freedom (1), expected frequencies are greater than 5=>we can trust results, fail to reject H0=>there is a relationship.

CHI-SQUARE TEST

Stroke Vs Glucose_labels

```
myF1= df['stroke']
myF2= df['glucose_labels']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
```

stroke	0	1
glucose_labels		
1	3019	112
2	961	37
3	881	100

```
(74.18076312175809,
 7.795643250529761e-17,
 2,
 array([[2978.43268102, 152.56731898],
        [ 949.36947162,  48.63052838],
        [ 933.19784736,  47.80215264]]))
```

Chi-square(74.18), p-value (7.779e-17), degrees of freedom (2), expected frequencies are greater than 5=> we can trust results, fail to reject H0=>there is a relationship.

Stroke Vs Bmi_labels

```
myF1= df['stroke']
myF2= df['bmi_labels']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
```

stroke	0	1
bmi_labels		
1	341	2
2	1224	35
3	1334	75
4	1962	137

```
(38.51824910812336,
 2.1953680478046033e-08,
 3,
 array([[ 326.28630137,  16.71369863],
        [1197.65146771,  61.34853229],
        [1340.34227006,  68.65772994],
        [1996.71996086, 102.28003914]]))
```

Chi-square(38.51), p-value 2.19e-08), degrees of freedom (1), expected frequencies are greater than 5=> we can trust results, fail to reject H0=>there is a relationship.

CHI-SQUARE TEST

Stroke Vs Smoking_status

```
myF1= df['stroke']
myF2= df['smoking_status']
mycrosstable= pd.crosstab(myF2,myF1)
print(mycrosstable)
chiVal, pVal, df1, exp= chi2_contingency(mycrosstable)
chiVal, pVal, df1, exp
```

stroke	0	1
smoking_status		
0	747	42
1	815	70
2	885	45
3	2414	92

```
(25.760905375925446,
1.0702431639507865e-05,
3,
array([[ 750.55362035,  38.44637965],
       [ 841.87573386,  43.12426614],
       [ 884.68297456,  45.31702544],
       [2383.88767123, 122.11232877]]))
```

Chi-square(25.76),
p-value (1.07e-05),
degrees of freedom
(3), expected
frequencies are
greater than 5=>we
can trust results, fail
to reject H0=>there
is a relationship.

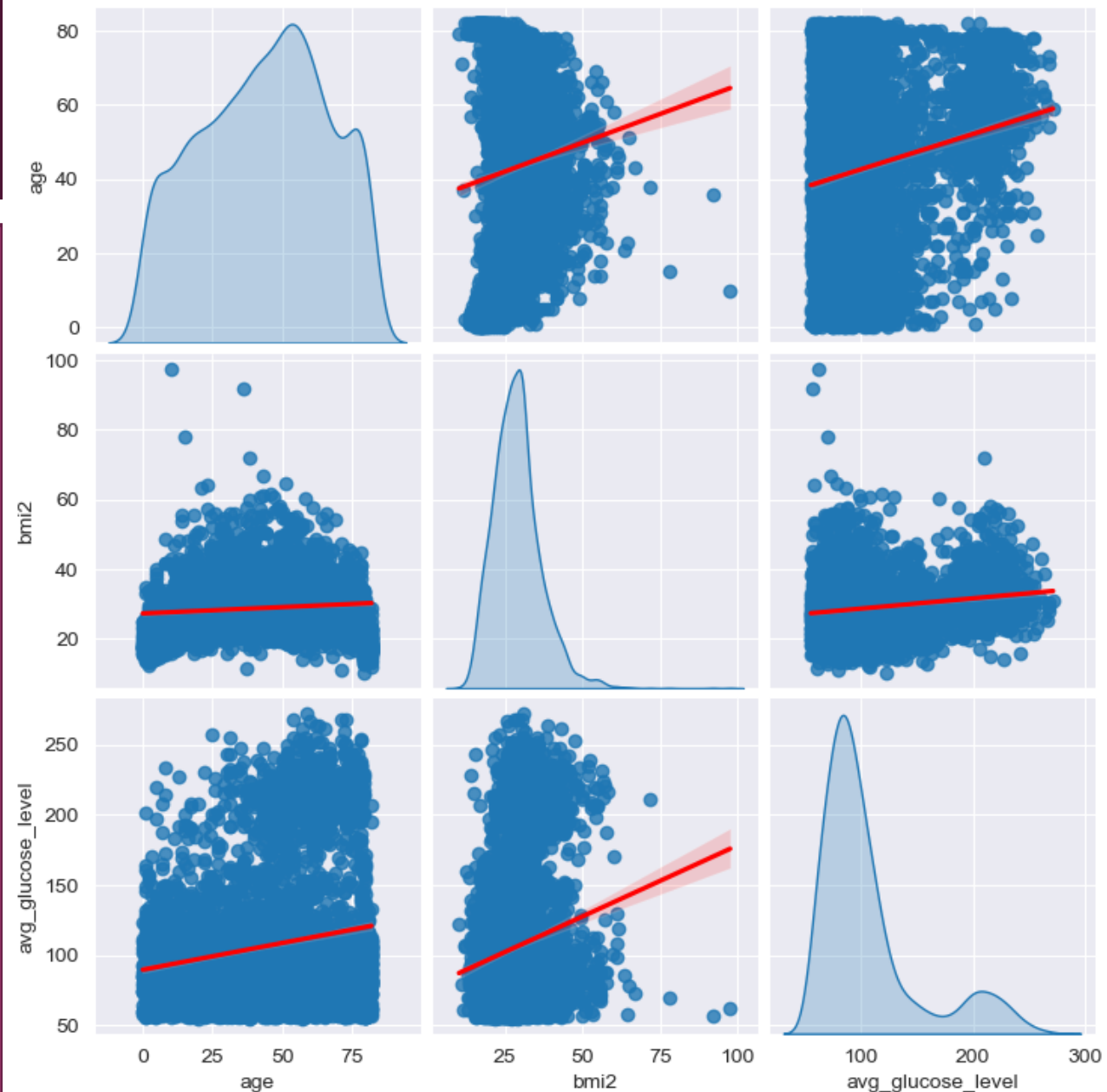
Percentage of each glucose_label whoever suffered stroke: 1-normal 3.6% 2-prediabetic 3.9% 3-diabetic 10.2% We see that percentage of patients with diabetics who suffered a stroke has the highest value. We can conclude that the first greatest dependency is between stroke and age (Chi_square=390.382). The second is stroke heart disease with a Chi-square value of 90.6. And the third is stroke and hypertension (Chi= square=81.6). The fourth greatest dependency is stroke and glucose level with the highest percentage (10.2%) of patients who suffered stroke falls into the diabetic group. The least dependency for the stroke and smokins_status.

REGRESSION

The pair plot multivariate analysis shows between age, BMI and average glucose level. All of them are continuous variables.

There is so significant correlation found. Except :

1. Age vs bmi and
2. Average glucose level vs bmi



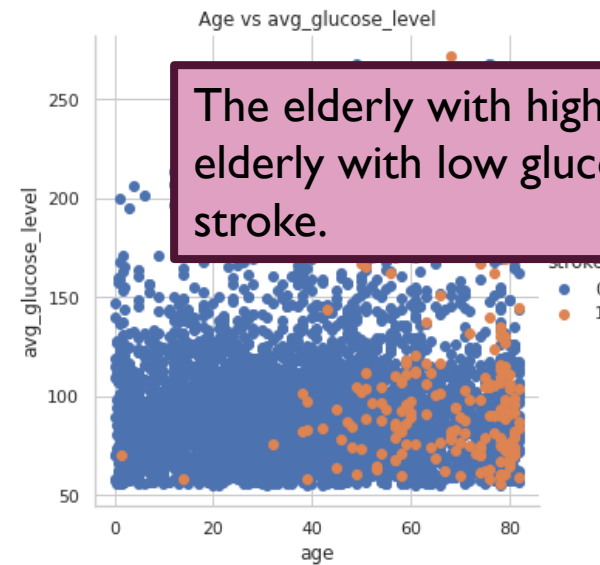
MULTIVARIATE

Stroke vs Gender vs Age



Stroke vs Glucose vs Age

```
sns.set_style("whitegrid")
sns.FacetGrid(df, hue="stroke", height=5).map(plt.scatter, "age", "avg_glucose_level").add_legend()
plt.title('Age vs avg_glucose_level')
plt.show()
```



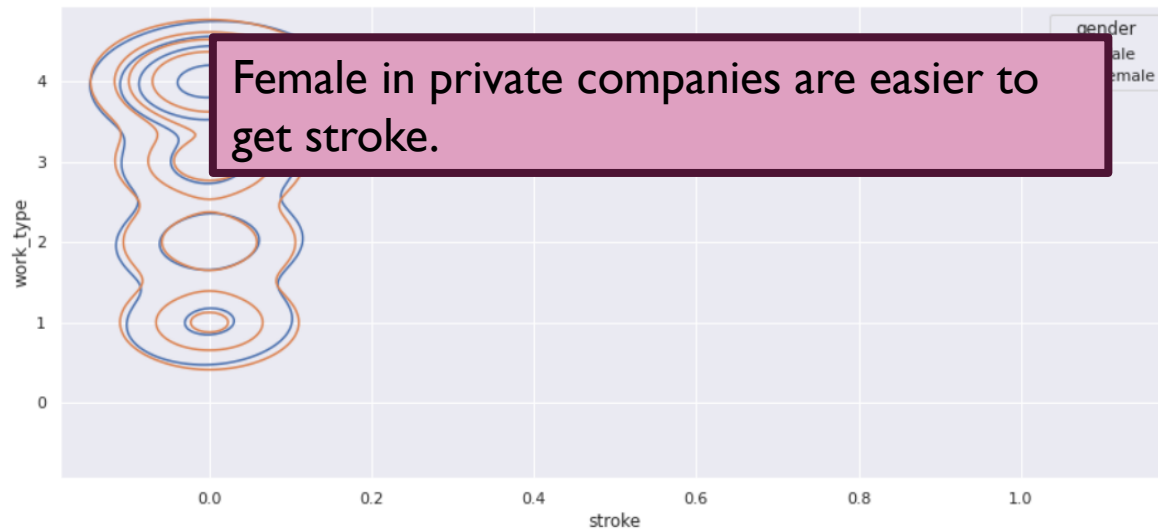
The elderly with high glucose and the elderly with low glucose are both prone to stroke.

MULTIVARIATE

Stroke vs Gender vs Work_type

```
df['work_type'].replace(['Never_worked', 'Govt_job', 'children', 'Self-employed', 'Private'],  
                        [0, 1, 2, 3, 4], inplace=True)  
  
sns.kdeplot(data=df, x="stroke", y='work_type', hue="gender", fill=False, levels=5, thresh=0.1)
```

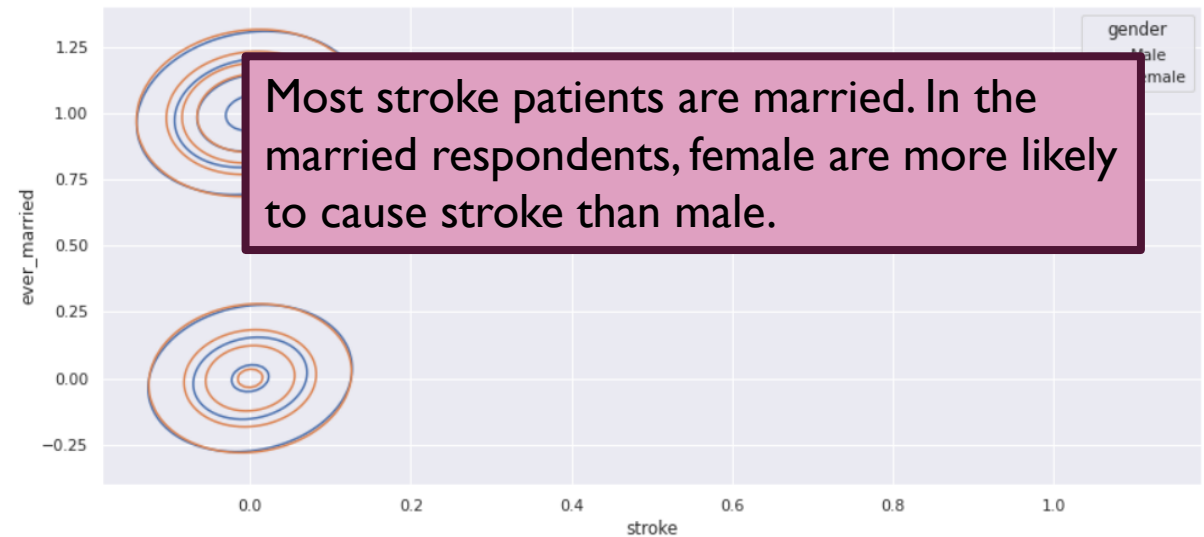
<AxesSubplot: xlabel='stroke', ylabel='work_type'>



Stroke vs Gender vs Ever_married

```
df['ever_married'].replace(['No', 'Yes'],  
                           [0, 1], inplace=True)  
  
sns.kdeplot(data=df, x="stroke", y='ever_married', hue="gender", fill=False, levels=5, thresh=0.1)
```

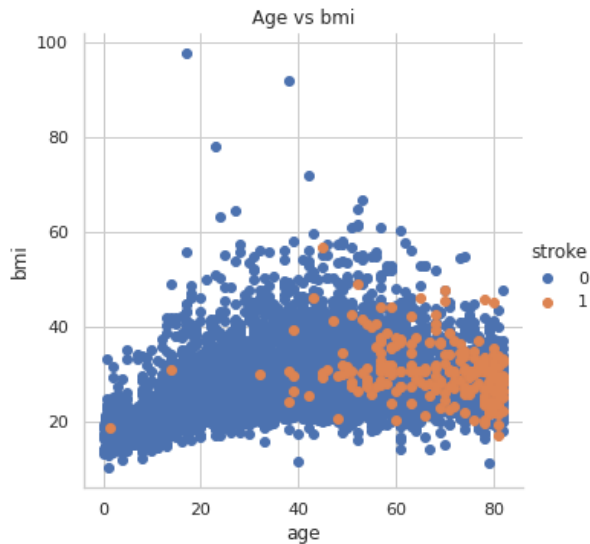
<AxesSubplot: xlabel='stroke', ylabel='ever_married'>



MULTIVARIATE

Stroke vs BMI vs Age

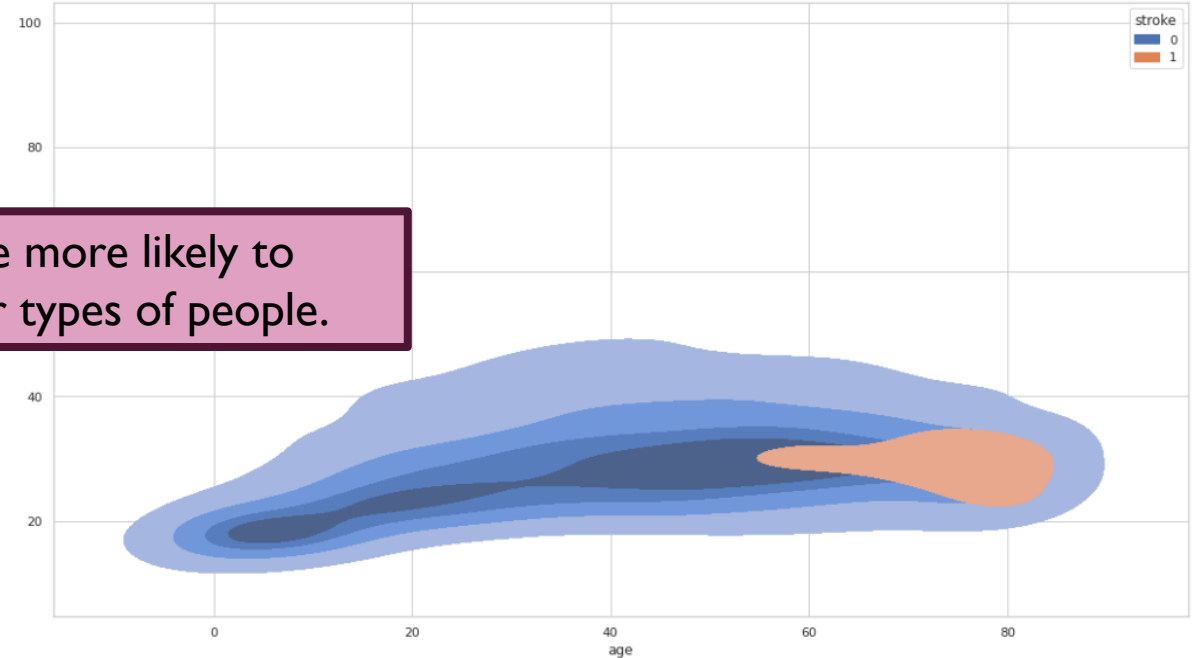
```
#Avg_glucose vs stroke vs gender #Layer 2 (Y)
# Layer 2 Bivariate analysis / avg_glu_level,age,stroke ASMA
sns.set_style("whitegrid")
sns.FacetGrid(df, hue="stroke", height=5).map(plt.scatter, "age", "bmi").add_legend()
plt.title('Age vs bmi')
plt.show()
```



Elderly obese people are more likely to have a stroke than other types of people.

```
sns.kdeplot(data=df, x="age", y='bmi', hue="stroke", fill=True, levels=5, thresh=0.1)
```

<AxesSubplot:xlabel='age', ylabel='bmi'>



MULTIVARIATE

Stroke vs Gender vs Residence_type

```
.kdeplot(data=df, x="stroke", y='Residence_type', hue="gender", fill=False, levels=5, thresh=0.
```

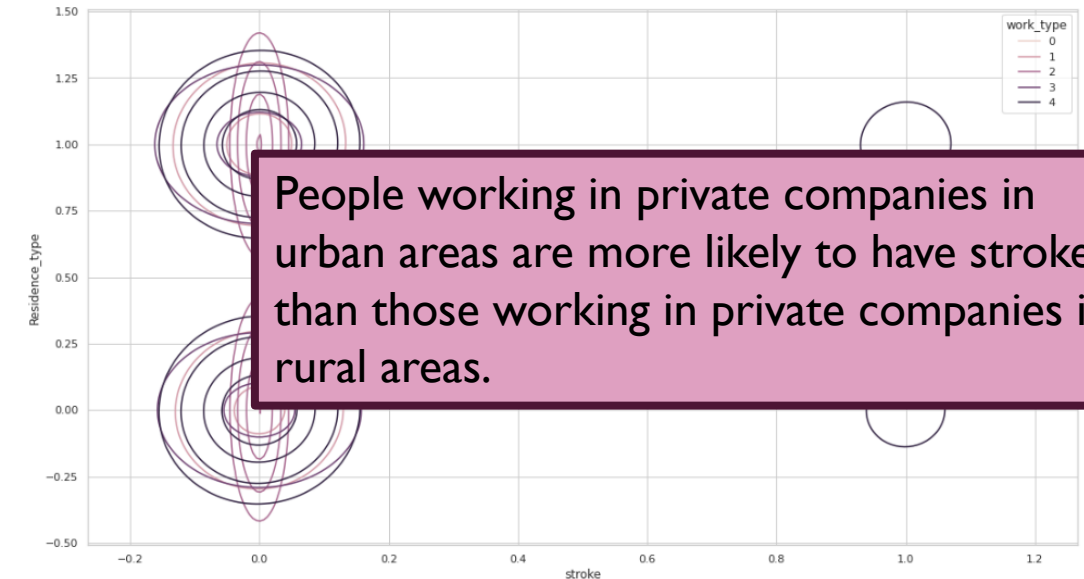
<AxesSubplot:xlabel='stroke', ylabel='Residence_type'>



Stroke vs Residence_type vs Work_type

```
deplot(data=df, x="stroke", y='Residence_type', hue="work_type", fill=False, levels=5, thresh=0
```

<AxesSubplot:xlabel='stroke', ylabel='Residence_type'>



MODEL PREPARATION

- Encoding categorical values
- `df['gender'] = df['gender'].replace({'Male':0,'Female':1,'Other':-1}).astype(np.uint8)`
- `df['Residence_type'] = df['Residence_type'].replace({'Rural':0,'Urban':1}).astype(np.uint8)`
- `df['work_type'] = df['work_type'].replace({'Private':0,'Self-employed':1,'Govt_job':2,'children':-1,'Never_worked':-2}).astype(np.uint8)`

LOGISTIC REGRESSION

Logistic Regression to Predict Stroke Using Different Variables

The dependent variable is stroke and has a zero (0) and (1) values:

The independent variables are age, BMI and average glucose level,

```
#Logistic Regression to predict stroke using different variables
training_df=df[['age', 'avg_glucose_level','bmi' , 'stroke']]
y= training_df['stroke']
x= training_df.drop(["stroke"],axis=1,inplace=False)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,random_state=16)

lr = LogisticRegression()

# fit the model with data
lr.fit(x_train,y_train)
test_predict=lr.predict(x_test)

print("predicted strokes" ,test_predict)
print(lr.predict([[45,100,30],[80,100,200],[20,56,18],[45,93.72,30.2],[66,151.16,27.5],[68,211.06,39.3],[100,402,80]])))

score = lr.score(x_test, y_test)
print("accuracy " ,score)

predicted strokes [0 0 0 ... 0 0 0]
[0 0 0 0 0 0 1]
accuracy 0.9491392801251957
```


CROSS VALIDATION MODEL

```
]:  
rf_cv = cross_val_score(rf_pipeline,X_train_res,y_train_res,cv=10,scoring='f1')  
svm_cv = cross_val_score(svm_pipeline,X_train_res,y_train_res,cv=10,scoring='f1')  
logreg_cv = cross_val_score(logreg_pipeline,X_train_res,y_train_res,cv=10,scoring='f1')
```

```
]:  
print('Mean f1 scores:')  
print('Random Forest mean :',cross_val_score(rf_pipeline,X_train_res,y_train_res,cv=10,scoring  
='f1').mean())  
print('SVM mean :',cross_val_score(svm_pipeline,X_train_res,y_train_res,cv=10,scoring='f1').me  
an())  
print('Logistic Regression mean :',cross_val_score(logreg_pipeline,X_train_res,y_train_res,cv=  
10,scoring='f1').mean())
```

Mean f1 scores:

Random Forest mean : 0.9342717632419655

SVM mean : 0.8752026263943018

Logistic Regression mean : 0.8225682495045643

CONCLUSION ABOUT THE MODEL

- Age variable was good indicator to predicting a stroke,
- Extensive visualization and multiple models conducted,
 - Random Forest,
 - Logistic Regression
- Despite Random Forest had the highest accuracy, Hyperparameter tuning on all models to improve their results, while, Tuned Logistic Regression model provided the best recall,
- **Therefore Tuned Logistic Regression is a better model**, despite Random Forest had the highest accuracy,

GENERAL FINDING - CONCLUSION

- 1. It seemed like both **BMI and Age were positively correlated**, though the association was not strong,
- 2. **Older patients are more likely to suffer a stroke than a younger patient.** And elderly obese people are more likely to have a stroke than other types of people,
- 3. **Higher BMI increases the stroke risk,**
- 4. The elderly with high glucose and the elderly with low glucose are both prone to stroke,
- 5. Female in private companies and self employment are easier to get stroke,
- 6. Most stroke patients are married. In the married population, women are more likely to cause stroke than men,
- 7. **Female living in cities is more likely to have strokes than male.** (More stroke cases are observed urban than rural),
- 8. **People working in private companies in urban areas are more likely to have strokes than those working in private companies in rural areas,**



Thank you !